The Transformer

The backbone of LLMs

Stephanie Brandl

Agenda

- Recap: language models, word2vec
- Transformers
  - Overview
  - Encoder-only: BERT
  - Decoder-only: GPT
- Self-attention

Agenda

This lecture is based on

“Speech and Language Processing”, 2025. Jurafsky & Martin

“Hands-On Large Language Models”, 2024. Alammar & Grootendorst

The Illustrated Transformer

Additional resources:

“Let’s build GPT from Scratch”, by Andrej Karpathy (more code-oriented)

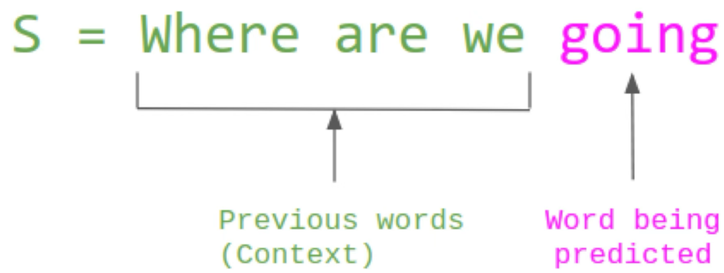
Stanford CS224N: NLP with Deep Learning

there are many more great resources!

Recap

What is a language model?

“A language model assigns a probability to each possible next word or gives a probability distribution over possible next words”



$$P(S) = P(\text{Where}) \times P(\text{are} \mid \text{Where}) \times P(\text{we} \mid \text{Where are}) \times P(\text{going} \mid \text{Where are we})$$

Jurafsky & Martin, 2025. Speech and Language Processing:
An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models (p. 32)

word2vec
Intuition

- static embeddings, one fixed embedding for each word in the vocabulary
- instead of counting co-occurrences, a classifier is trained on a binary prediction task: “Is word w likely to appear next to word t?”
- we don’t really care about the prediction but we learn embeddings while doing this
- we train this on running text in a self-supervised manner: no need for hand-labeling features (similar to tf-idf)

word2vec
In a nutshell

1. Treat the target word and a neighboring context word as positive examples
2. Randomly sample other words in the lexicon to get negative samples
3. Use logistic regression to train a classifier to distinguish those two cases
4. Use the learned weights as embeddings

Evaluating embeddings

Queen is to King as Women is to ❓

a is to b as a* is to what?

a : b :: a* : b*

$$\tilde{\mathbf{b}}^* = \arg\min_x \text{distance}(\mathbf{x}, \mathbf{b} - \mathbf{a} + \mathbf{a}^*)$$

Some examples:

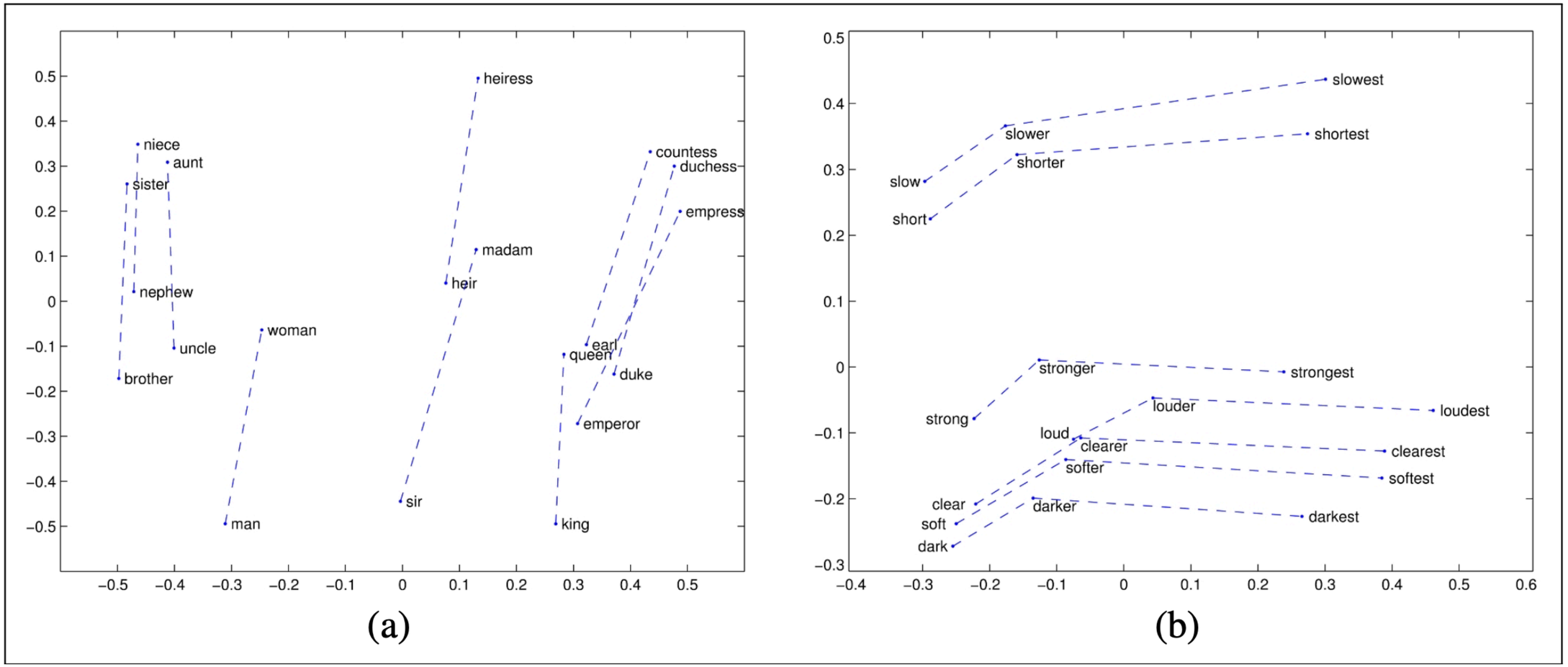
queen:king :: woman: ?

Paris:France :: Rome: ?

big:bigger :: small: ?

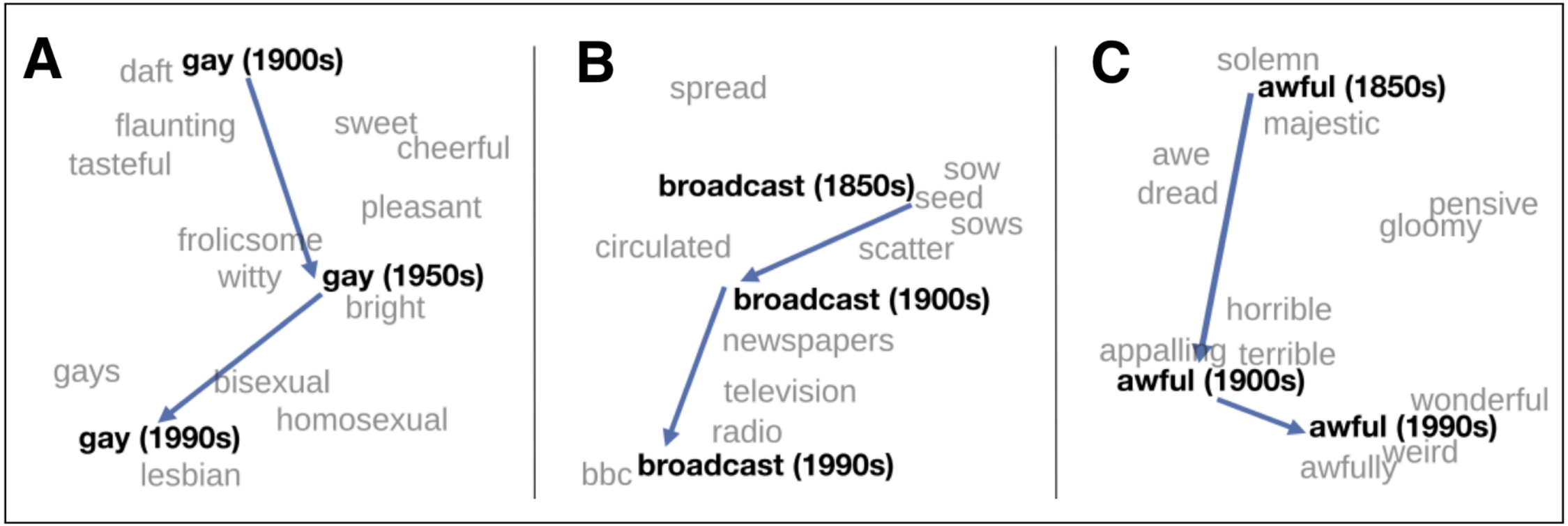
Visualizing embeddings

Trajectories in 2D



Visualizing embeddings

Trajectories in 2D



Transformers

The Transformer Architecture

Encoder-Decoder

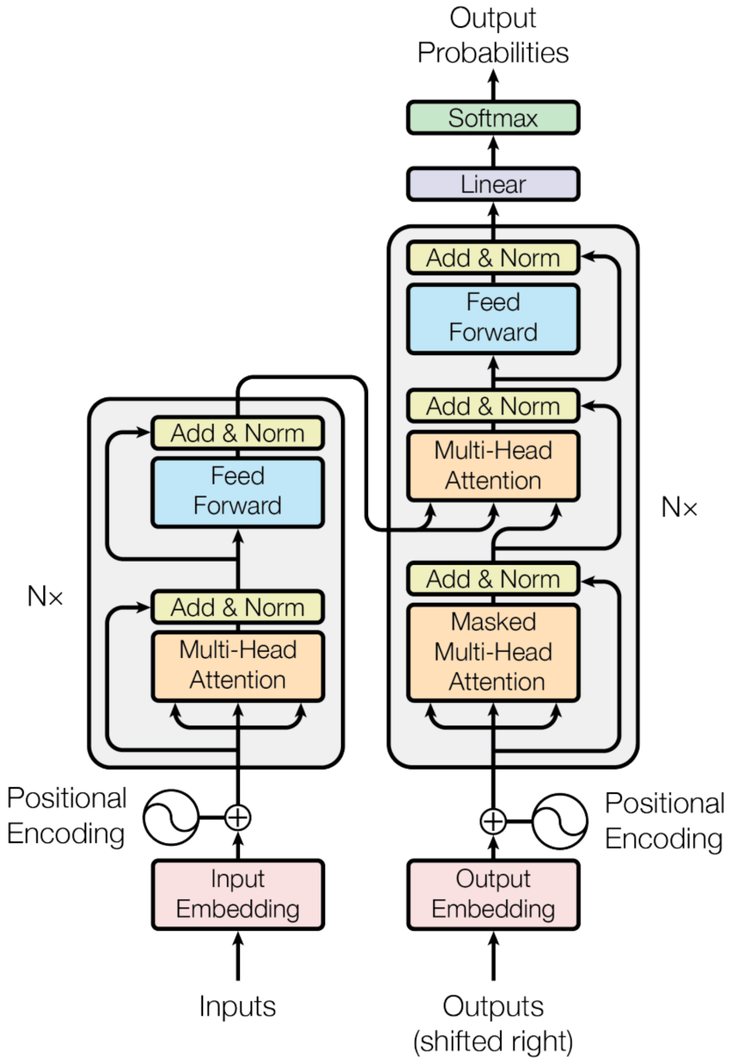


Figure 1: The Transformer - model architecture.

- Introduced in 2017 by Vaswani et al.
- Rose to prominence through BERT and GPT
- Initiated the era of pre-trained language models
- Transformers are still used in LLMs like GPT4 or Llama

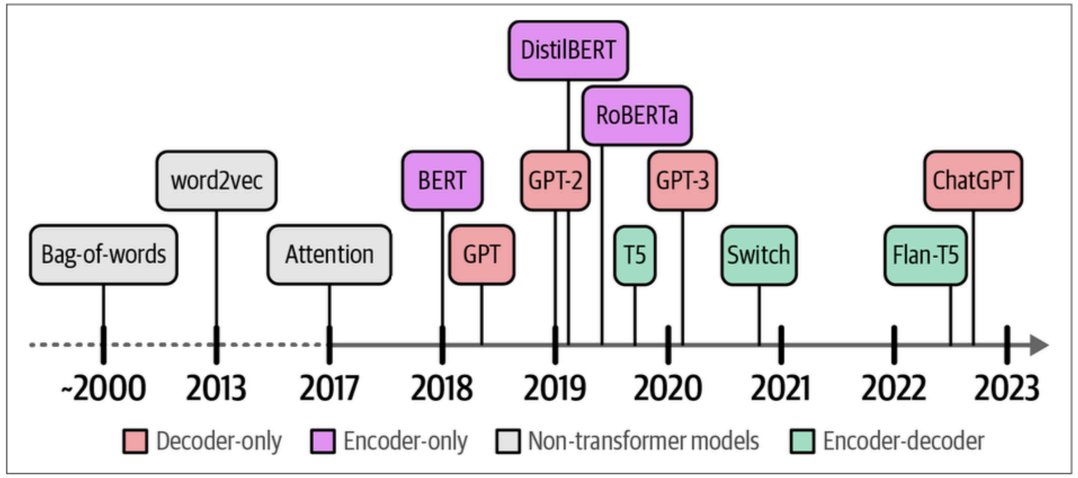


Figure 1-1. A peek into the history of Language AI.

Attention Is All You Need (Vaswani et al., NeurIPS 2017)
Improving language understanding by generative pre-training (Radford et al., 2018)
BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding (Devlin et al., NAACL 2019)

The Transformer Architecture

Encoder-Decoder

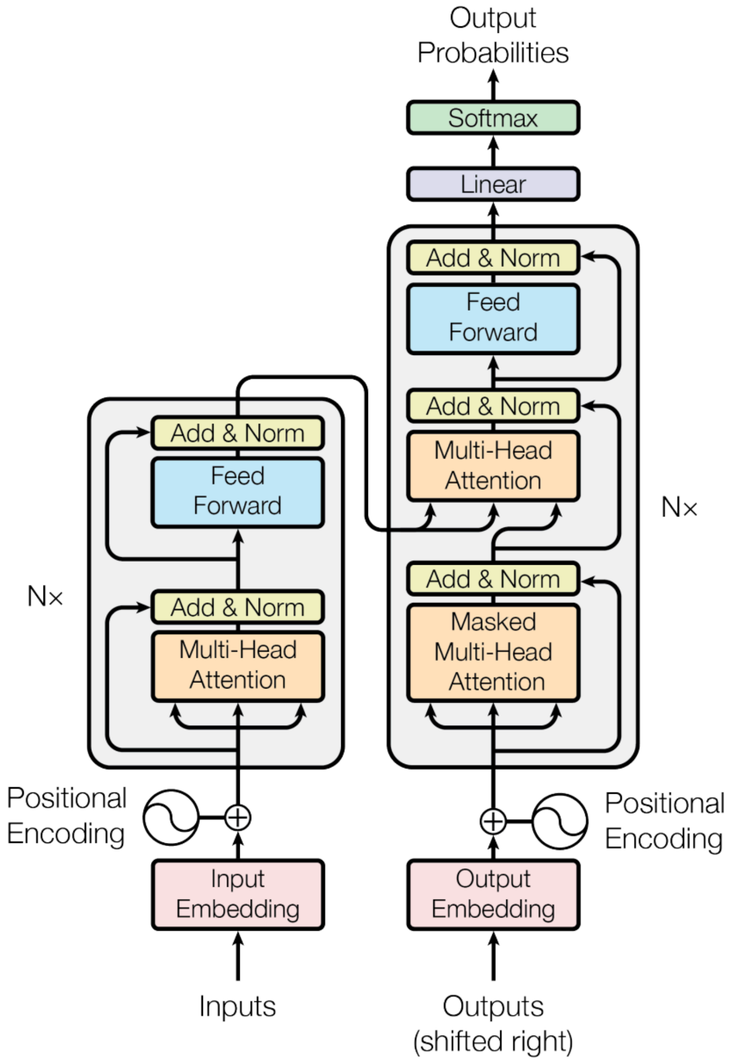


Figure 1: The Transformer - model architecture.

What's new?

- (multi-head) attention at the core: tokens can "communicate" with each other
- parallelization: enormous speed-up and room for bigger models
- foundation models: one big multi-purpose model is trained and released and only needs to be "fine-tuned" for specific tasks

The Transformer Architecture

Encoder-Decoder

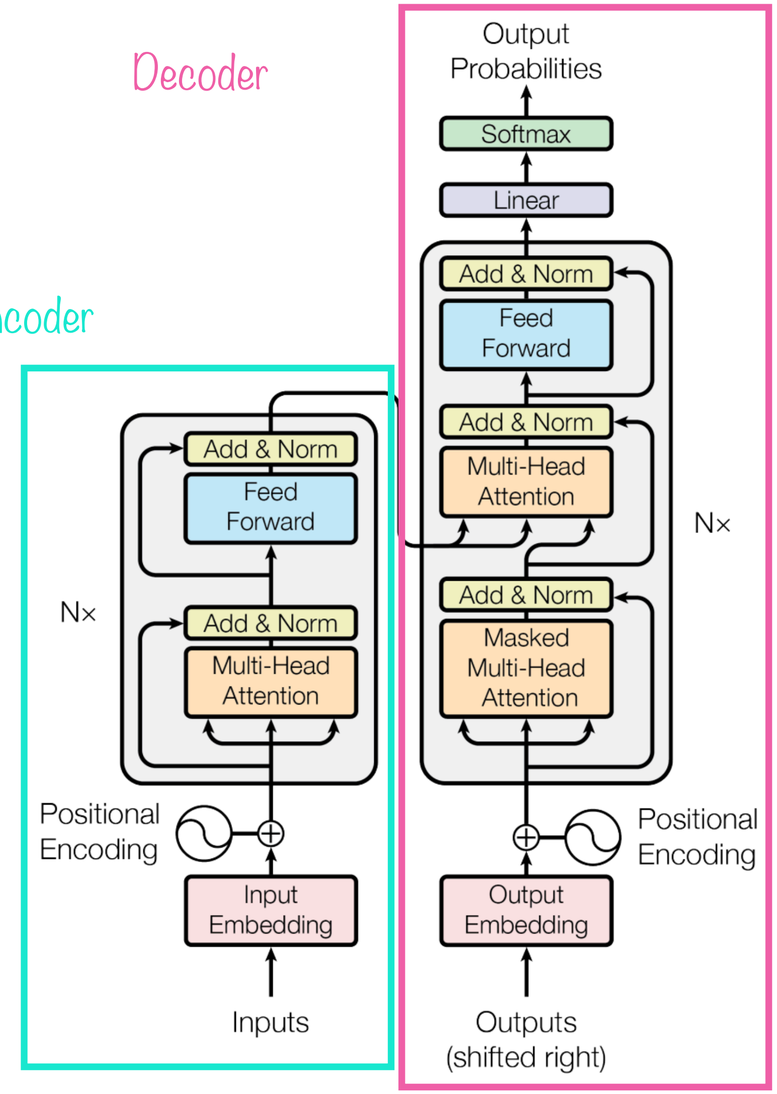


Figure 1: The Transformer - model architecture.

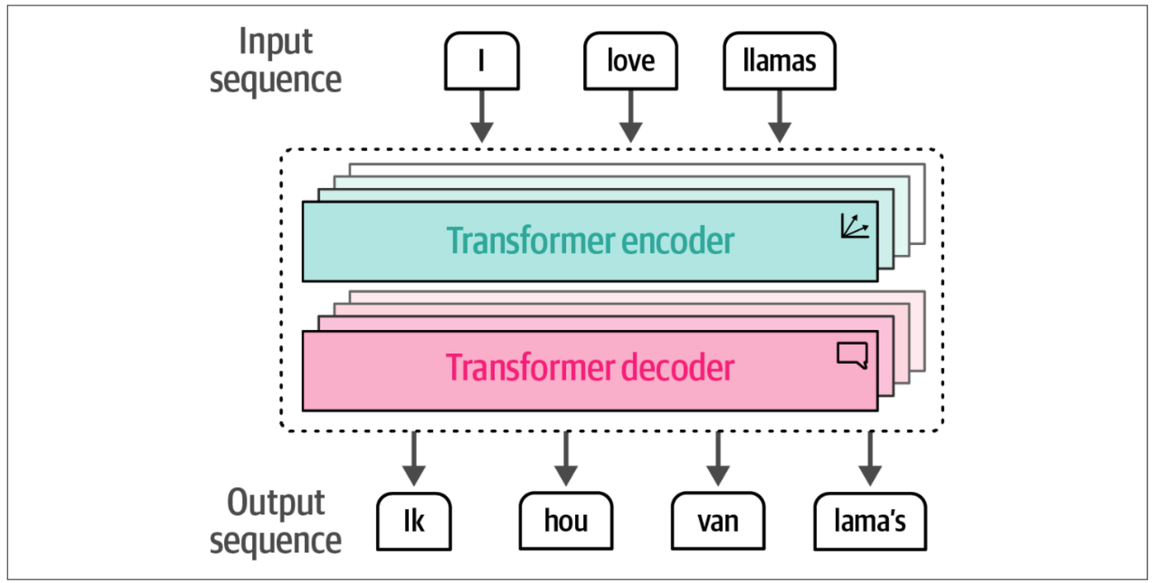


Figure 1-16. The Transformer is a combination of stacked encoder and decoder blocks where the input flows through each encoder and decoder.

Encoder-Decoder Models can be used for translation:
The encoder "encodes" the sentence in the original language, passes the learned (numerical!) representations to the decoder which "decodes" the sentence into the target language

BERT: Bidirectional Encoder Representations from Transformers

Encoder-only

Encoder

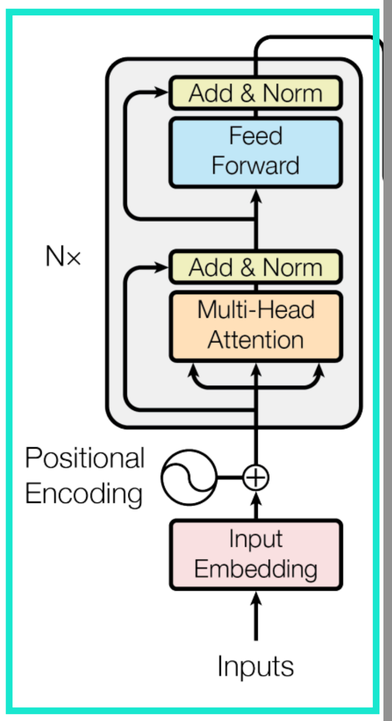


Figure 1: The Transformer - model architecture.

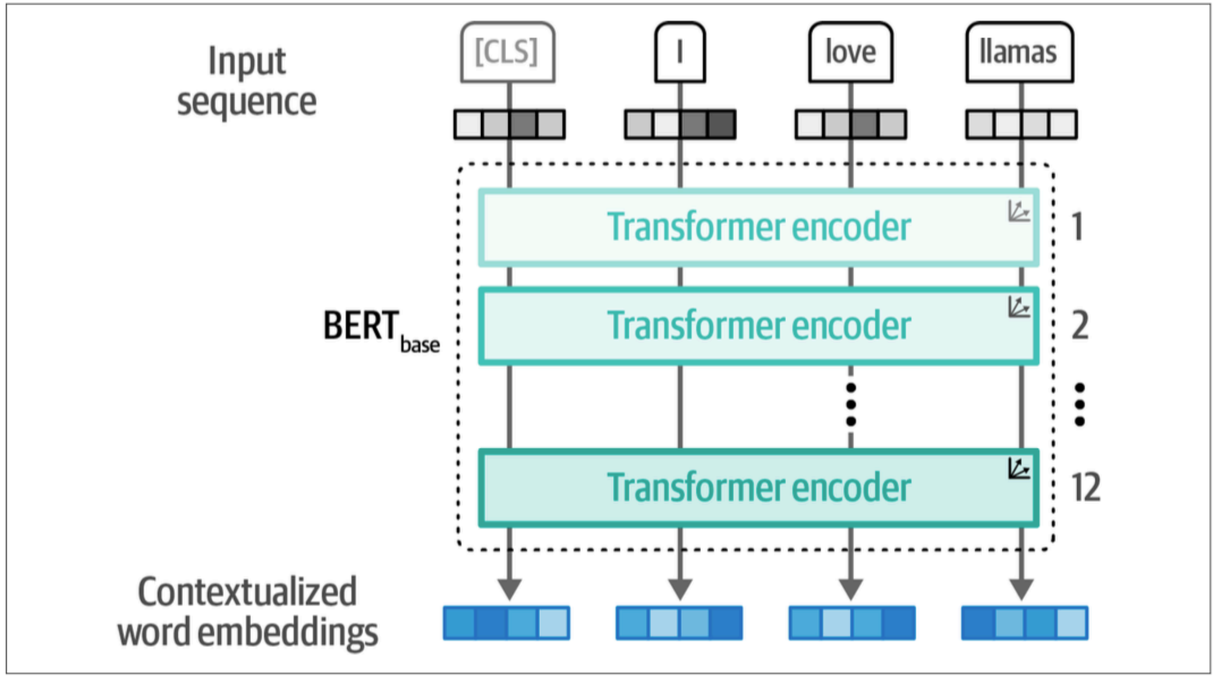


Figure 1-21. The architecture of a BERT base model with 12 encoders.

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding (Devlin et al., NAACL 2019)

BERT: Bidirectional Encoder Representations from Transformers

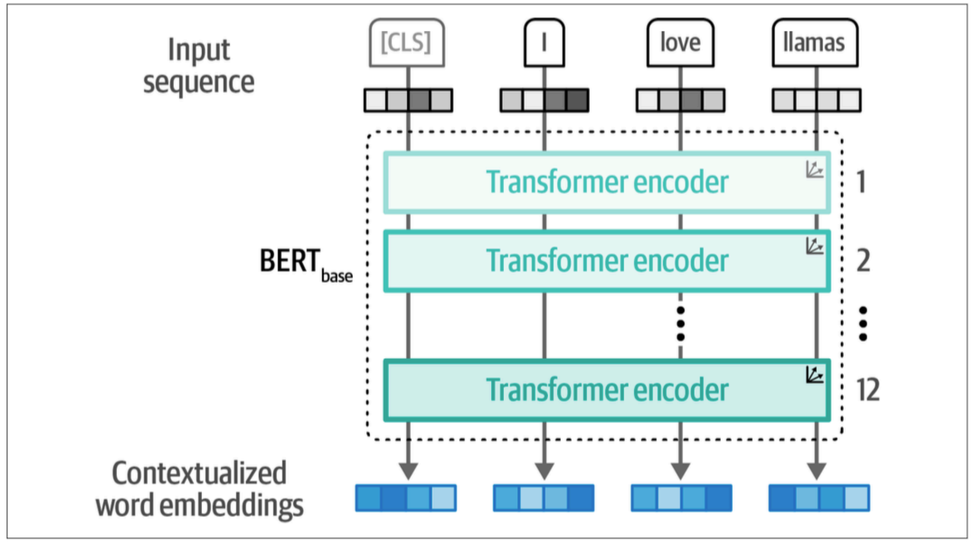


Figure 1-21. The architecture of a BERT base model with 12 encoders.

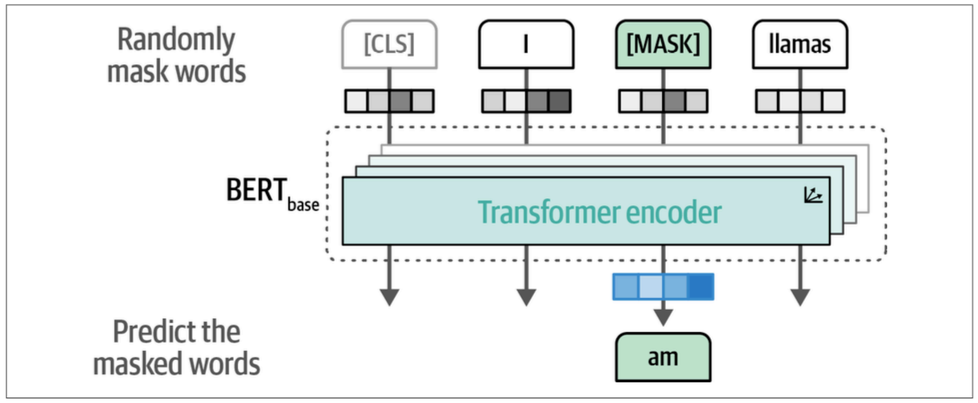
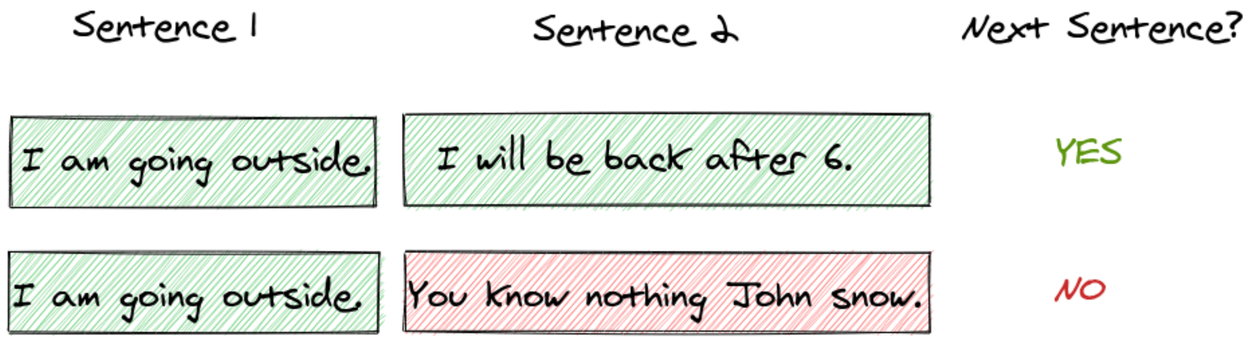


Figure 1-22. Train a BERT model by using masked language modeling.

- encoder-only
- bidirectional context: considers context from past and future tokens (similar to word2vec)
- pre-trained with masked-language modelling and next-sentence prediction
  - masked-language modelling: 15% of all tokens are randomly replaced with either a [MASK] token or a random token
  - next-sentence prediction: binary classification task if two sequences appeared sequentially



example for next-sentence prediction

BERT: Bidirectional Encoder Representations from Transformers

- encoder-only
- bidirectional context: considers context from past and future tokens (similar to word2vec)
- pre-trained with masked-language modelling and next-sentence prediction
  - masked-language modelling: 15% of all tokens are randomly replaced with either a [MASK] token or a random token
  - next-sentence prediction: binary classification task if two sequences appeared sequentially
- outputs contextualized word embeddings:
  - word embeddings that change based on context: the same word can have multiple embeddings
  - these can then be used to train a task-specific model: fine-tuning

Finetuning BERT

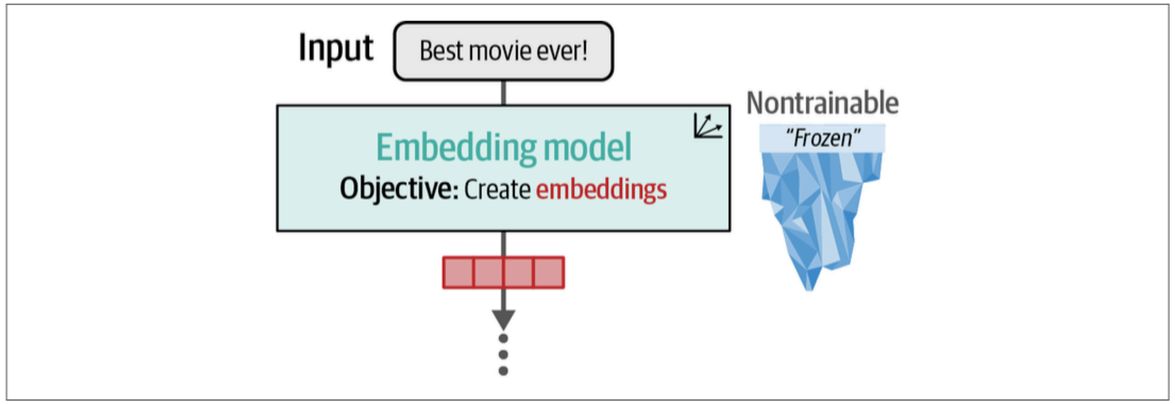


Figure 4-11. In step 1, we use the embedding model to extract the features and convert the input text to embeddings.

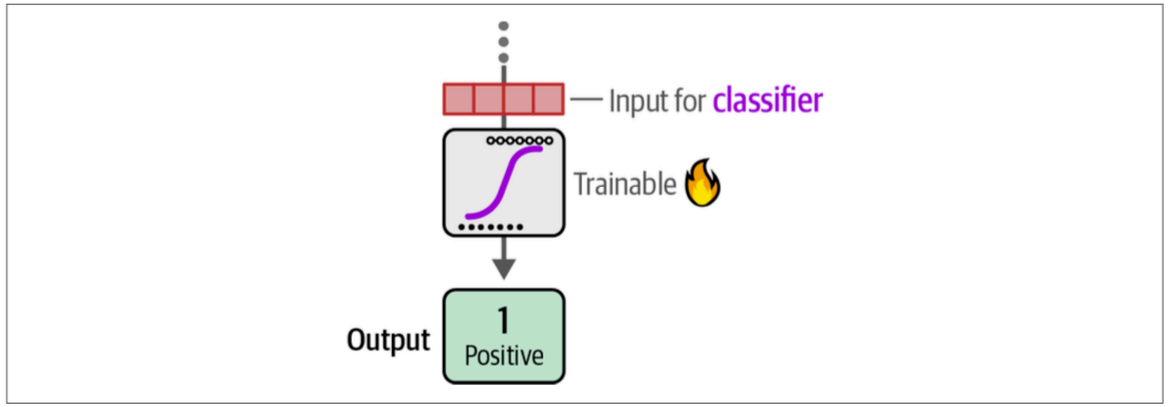


Figure 4-12. Using the embeddings as our features, we train a logistic regression model on our training data.

We can use the output embeddings of an encoder model to train a classifier, e.g., for sentiment classification

GPT: Generative Pre-trained Transformer

Decoder-only

Decoder

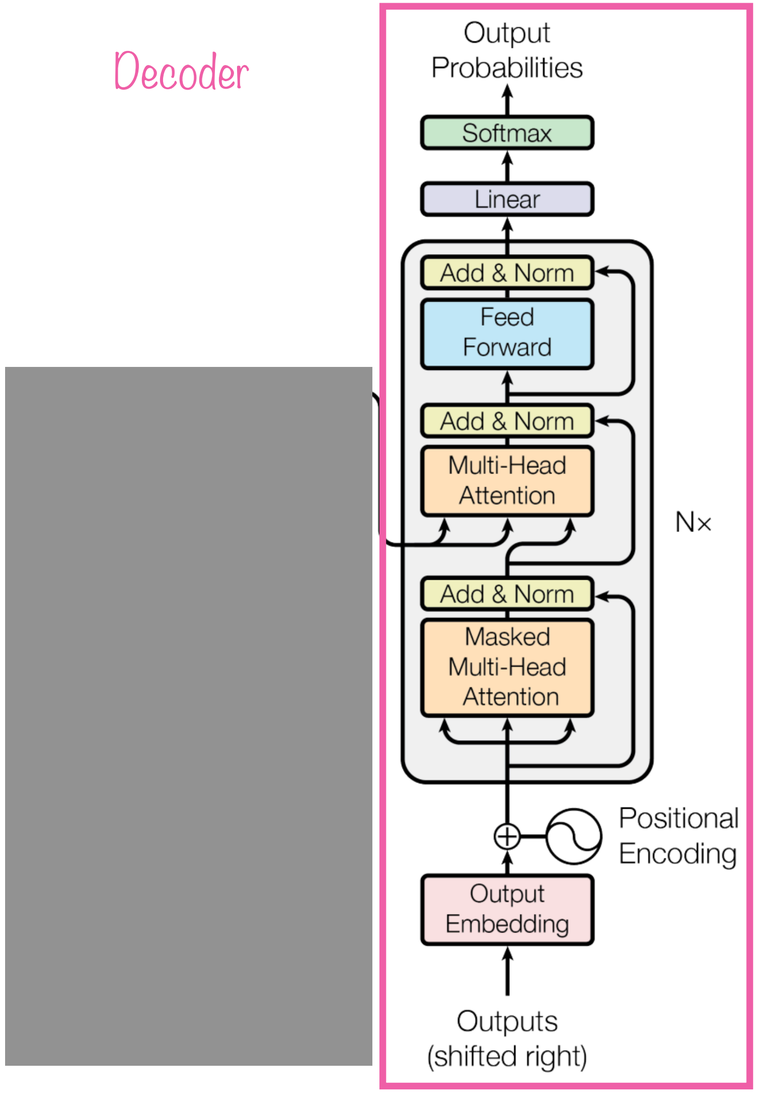


Figure 1: The Transformer - model architecture.

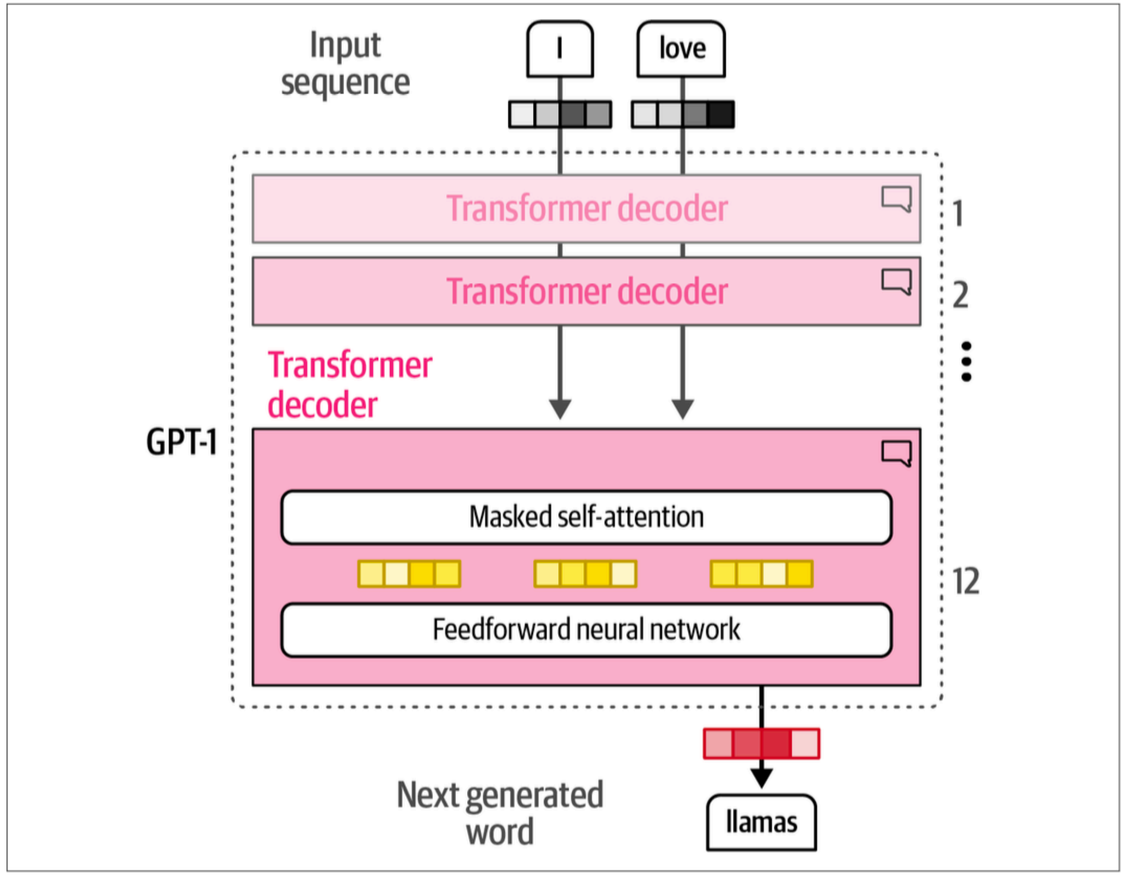


Figure 1-24. The architecture of a GPT-1. It uses a decoder-only architecture and removes the encoder-attention block.

Improving language understanding by generative pre-training (Radford et al., 2018)

GPT: Generative Pre-trained Transformer

Decoder-only

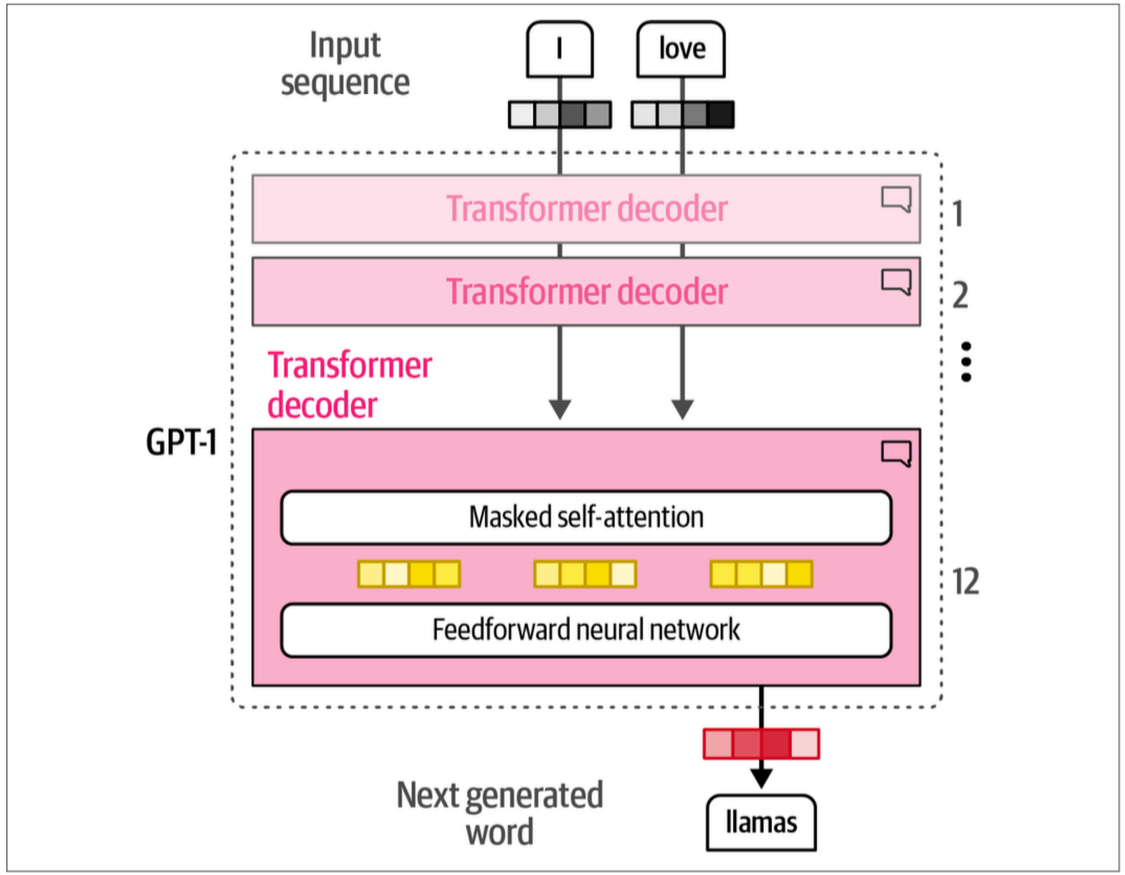


Figure 1-24. The architecture of a GPT-1. It uses a decoder-only architecture and removes the encoder-attention block.

- decoder-only
- unidirectional context: only considers previous tokens (similar to speech, text)
- autoregressive: trained to predict next tokens sequentially → start of generative AI 💥
- pre-trained on next-token prediction
- can be fine-tuned for specific tasks, similar to BERT
- we will focus on decoder-only language models in this lecture

Finetuning GPT

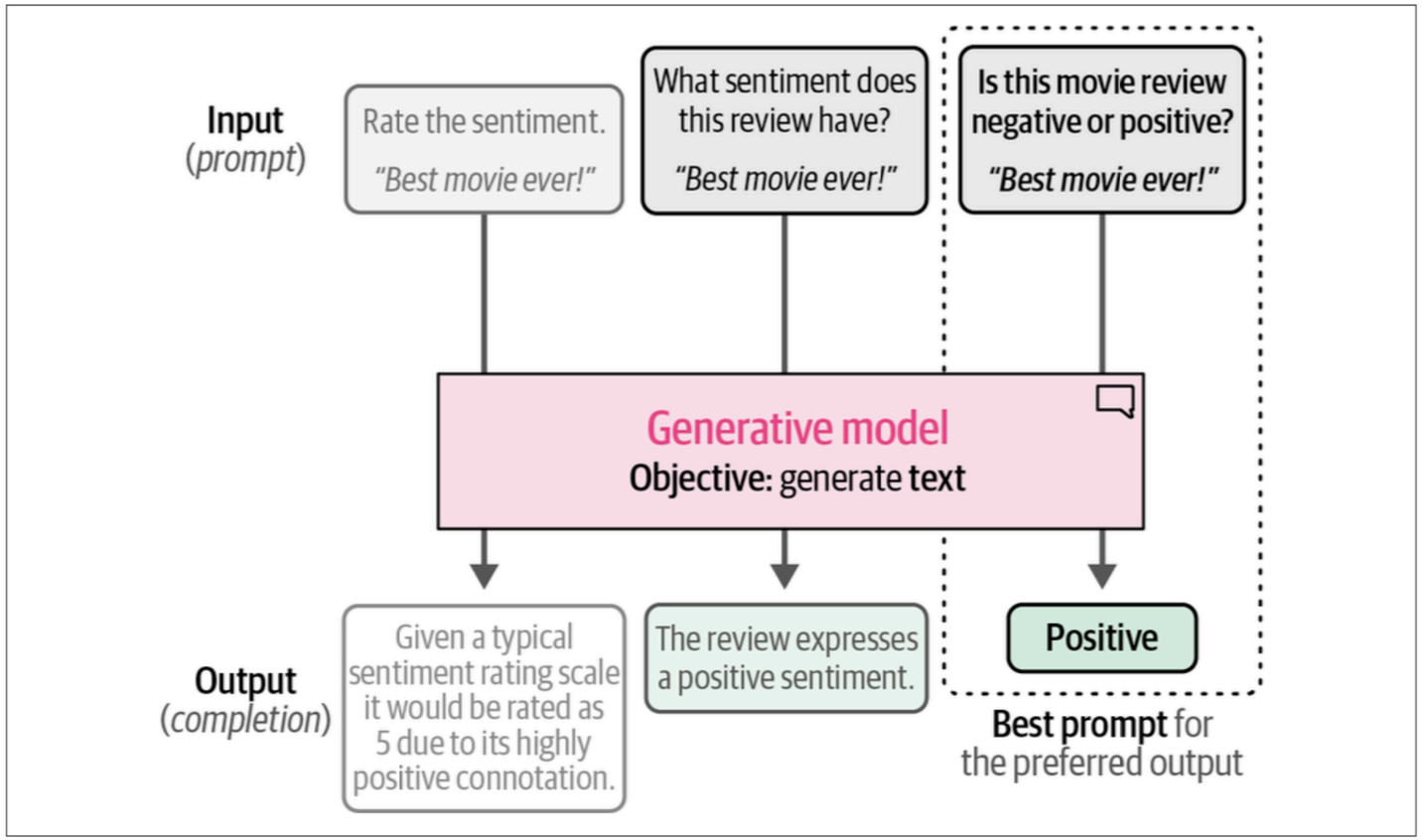


Figure 4-18. Prompt engineering allows prompts to be updated to improve the output generated by the model.

🍕 Take Aways

Transformers...

- allow for faster and more complex language modelling through parallelization and unsupervised pre-training:
  - most computations can be parallelized to speed-up computation
  - no labels are needed for masked-language modelling, next-sentence prediction or next-token prediction, everything can be extracted from existing text
- general purpose language models: “foundation models” that only need fine-tuning to solve specific tasks
- BERT an encoder-only language model that outputs contextualized word embeddings, considers previous and following tokens
- GPT a decoder-only language model (autoregressive) that generates next tokens sequentially, only considers previous tokens → beginning of genAI ✨
- at the heart of this lies the self-attention mechanism 👀

Self-Attention

Static Word Embeddings

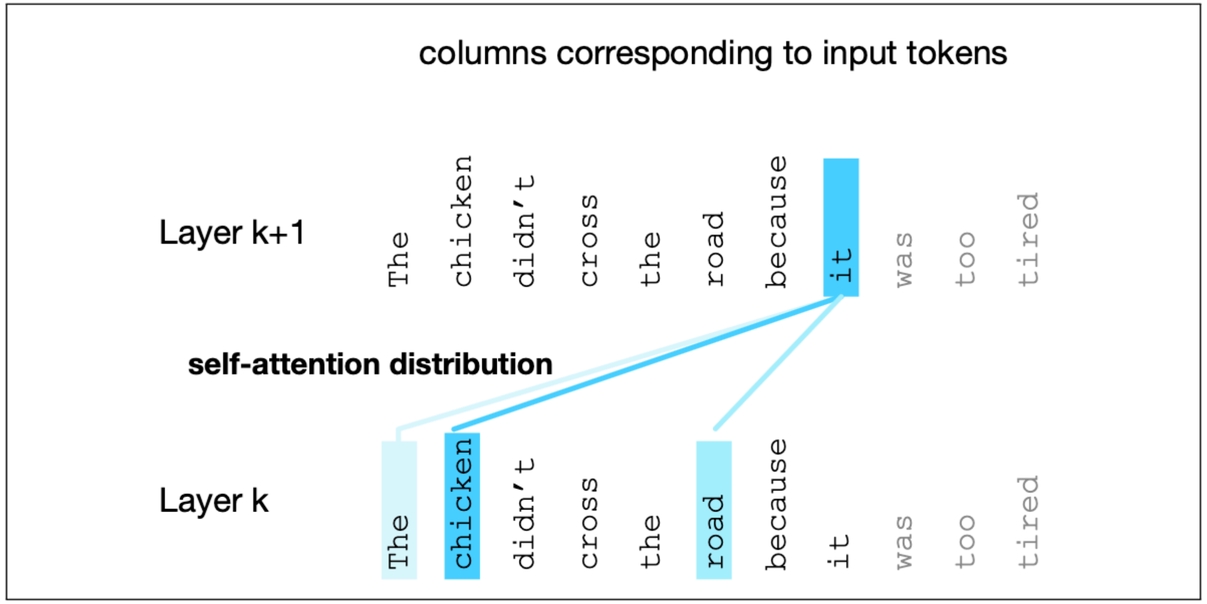
- Recap: word2vec learns a single static representation/embedding for each word in the vocabulary
- Let's consider the word ***it***:
  - The chicken didn't cross the road because ***it*** was too tired
  - The chicken didn't cross the road because ***it*** was too wide
- What does ***it*** refer to in those two sentences?
  - The ***chicken*** didn't cross the road because ***it*** was too tired
  - The chicken didn't cross the ***road*** because ***it*** was too wide
- Static word representations are not able to capture this difference, ***it*** always carries the same information

Contextual Word Embeddings

- Contextual words, i.e., the words around a target word, help us to understand or compute the meaning
  - The ***chicken*** didn't cross the road because ***it*** was too ***tired***
  - The chicken didn't cross the ***road*** because ***it*** was too ***wide***
- Relevant contextual information can be "far" away from the target word, remember that we used a window size of 5 words for word2vec
- Transformers are able to build contextual word embeddings by integrating the meaning of relevant contextual words
- layer by layer, richer and richer contextual representations of the input words are learned
- at each layer, the representation of a word is updated by combining information from previous layers with information about neighbouring tokens

Self-attention

- self-attention is the mechanism behind contextual word embeddings
- it weighs and combines the representation of relevant tokens in the context from layer $k-1$ to build the representation for tokens in layer k
- it allows tokens to “communicate” with each other



Example for attention weights

Self-attention

⚠️ Simplified Formalisation ⚠️

- attention takes as input a representation vector $\mathbf{x}_i$, corresponding to the input token at position i and a context window of prior inputs $\mathbf{x}_1 \ldots \mathbf{x}_{i-1}$ and produces the attention output $\mathbf{a}_i$
- in autoregressive or causal models, the context is any of the previous words
- at the end, attention is just a weighted sum of context words
- computing those weights is the complex part, but let's start with a simplified version

$$\text{Simplified version: } \mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{x}_j$$

where $\alpha_{i,j}$ is a scalar that weighs the value of $\mathbf{x}_j$ to compute $\mathbf{a}_i$

Self-attention

⚠️ Simplified Formalisation ⚠️

Simplified version: $\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{x}_j$

where $\alpha_{i,j}$ is a scalar that weighs the value of $\mathbf{x}_j$ to compute $\mathbf{a}_i$

- How to compute $\alpha_{i,j}$?
  - we weight each prior embedding proportionally to how similar it is to the current token
  - similarity is computed via the dot product and normalized with a softmax to create a weight vector $\alpha_{i,j}, j \leq i$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j$$

$$\alpha_{i,j} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \ \forall j \leq i$$

  - ⚠️ Reminder: softmax is a function $\sigma : \mathbb{R}^K \to (0,1)^K$ that transforms a tuple $\mathbf{z} = (z_1 \ldots z_K) \in \mathbb{R}^K$ into a probability distribution

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

Self-attention

⚠️ Simplified Formalisation ⚠️: Example

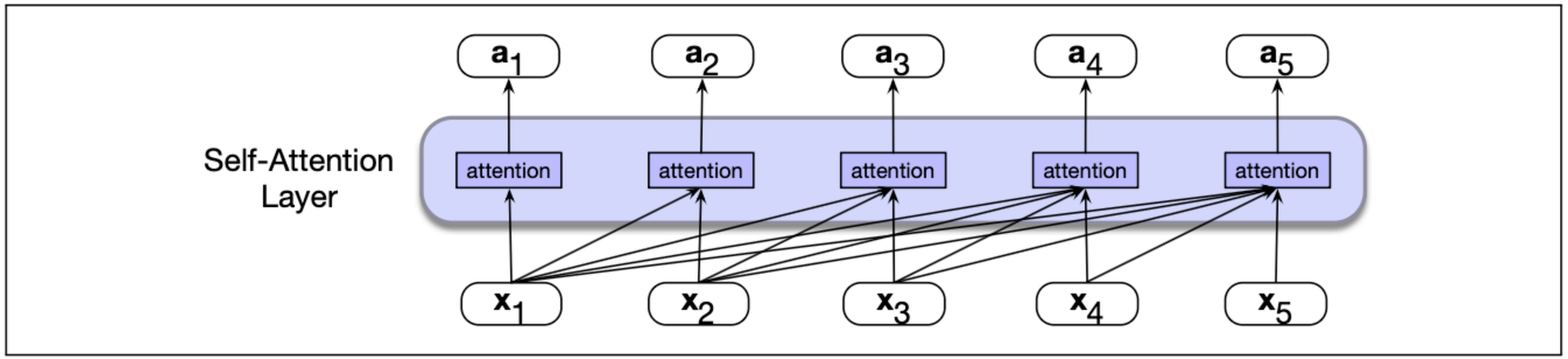


Figure 9.3 Information flow in causal self-attention. When processing each input $\mathbf{x}_i$, the model attends to all the inputs up to, and including $\mathbf{x}_i$.

$$\mathbf{a}_3 = \sum_{j \leq 3} \alpha_{3:} \mathbf{x}_j = \alpha_{31}\mathbf{x}_1 + \alpha_{32}\mathbf{x}_2 + \alpha_{33}\mathbf{x}_3$$

$$\alpha_{3:} = \text{softmax}(\mathbf{x}_3 \cdot \mathbf{x}_1, \mathbf{x}_3 \cdot \mathbf{x}_2, \mathbf{x}_3 \cdot \mathbf{x}_3)$$

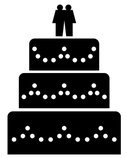
➡️ compare $\mathbf{x}_i$ to prior vectors, normalize into a probability distribution and use that to weigh the sum of the prior vector

Self-attention

The actual formalisation: query, key and value

that's how we call those layers

- each input embedding/word takes 3 different roles in an attention head
- from the perspective of a word/token:
  - **query**: "What am I looking for?"
  - **key**: "What do I contain?"
  - **value**: That is what we aggregate and weight in the end



Think of it as matchmaking:
"What am I looking for?"
"What do I offer?"
if what word_a and word_b are looking for/offer aligns ✨ it's a match 🫶

- query, key, value come with their own weight matrices: $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V$
- those matrices are learned in the training process
- each input embedding is projected by each of those matrices into the query, key and value space

$$\mathbf{q}_i = \mathbf{x}_i\mathbf{W}^Q;\ \ \mathbf{k}_i = \mathbf{x}_i\mathbf{W}^K;\ \ \mathbf{v}_i = \mathbf{x}_i\mathbf{W}^V$$

Self-attention

The actual formalisation: query, key and value

- remember from the simplified version:

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j$$
$$\alpha_{i,j} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \ \forall j \leq i$$

- now, instead we use the keys and the queries to compute the scores for the i-th token

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \ \mathbf{k}_j = \mathbf{x}_j \mathbf{W}^K; \ \mathbf{v}_j = \mathbf{x}_j \mathbf{W}^V$$
$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$
$$\alpha_{i,j} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \ \forall j \leq i$$

- d_k is the size of the query and key vectors, we normalize to avoid large values which can lead to numerical issues

Self-attention

The actual formalisation: query, key and value

$$\mathbf{x}_i \in \mathbb{R}^d;\ \mathbf{W}^Q, \mathbf{W}^K \in \mathbb{R}^{d \times d_k};\ \mathbf{W}^V \in \mathbb{R}^{d \times d_v}$$

$$\mathbf{q}_i = \mathbf{x}_i^\top \mathbf{W}^Q;\ \mathbf{k}_j = \mathbf{x}_j^\top \mathbf{W}^K;\ \mathbf{v}_j = \mathbf{x}_j^\top \mathbf{W}^V$$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i^\top \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

$$\alpha_{i,j} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j))\ \forall j \leq i$$

$$\mathbf{z}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{v}_j$$

Self-attention

Let's look at the dimensions, $\mathbf{a}_i$ needs to be of the same dimensionality than $\mathbf{x}_i$

$\mathbf{x}_i \in \mathbb{R}^{d \times 1}$

$\mathbf{x}_i \in \mathbb{R}^d; \ \mathbf{W}^Q, \mathbf{W}^K \in \mathbb{R}^{d \times d_k}; \ \mathbf{W}^V \in \mathbb{R}^{d \times d_v}$

$$\mathbf{q}_i = \mathbf{x}_i^\top \mathbf{W}^Q; \ \mathbf{k}_j = \mathbf{x}_j^\top \mathbf{W}^K; \quad \mathbf{v}_j = \mathbf{x}_j^\top \mathbf{W}^V$$

$\mathbf{v}_j \in \mathbb{R}^{d_v}$

$\mathbf{q}_i, \mathbf{k}_j \in \mathbb{R}^{d_k}$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i^\top \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

$\text{score}(\mathbf{x}_i, \mathbf{x}_j) \in \mathbb{R}$

$$\alpha_{i,j} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \ \forall j \leq i$$

$$\mathbf{z}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{v}_j$$

$\mathbf{z}_i \in \mathbb{R}^{d_v}$ ⚡

$$\mathbf{a}_i = \mathbf{z}_i^\top \mathbf{W}^O$$

we add a last projection,
to end up with the right dimensions

with $\mathbf{W}^O \in \mathbb{R}^{d_v \times d}$, so that $\mathbf{a}_i \in \mathbb{R}^d$

Self-attention
multi-head attention

These are the computations for a single attention head, in reality Transformers use multiple attention heads in each layer. So we have several separate attention heads, all with their own query, key, value matrices 🤯

$$\mathbf{x}_i \in \mathbb{R}^d;\ \mathbf{W}^Q, \mathbf{W}^K \in \mathbb{R}^{d \times d_k};\ \mathbf{W}^V \in \mathbb{R}^{d \times d_v}$$

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q;\ \mathbf{k}_j = \mathbf{x}_j \mathbf{W}^K;\ \mathbf{v}_j = \mathbf{x}_j \mathbf{W}^V$$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

$$\alpha_{i,j} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j))\ \forall j \le i$$

$$\mathbf{z}_i = \sum_{j \le i} \alpha_{ij} \mathbf{v}_j$$

$$\mathbf{a}_i = \mathbf{z}_i \mathbf{W}^O$$

Single-head attention

$$\mathbf{x}_i \in \mathbb{R}^d;\ \mathbf{W}^{Q_c}, \mathbf{W}^{K_c} \in \mathbb{R}^{d \times d_k};\ \mathbf{W}^{V_c} \in \mathbb{R}^{d \times d_v}$$

$$\mathbf{q}_i^c = \mathbf{x}_i \mathbf{W}^{Q_c};\ \mathbf{k}_j^c = \mathbf{x}_j \mathbf{W}^{K_c};\ \mathbf{v}_j^c = \mathbf{x}_j \mathbf{W}^{V_c};\quad \forall c\ 1 \le c \le A$$

$$\text{score}^c(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i^c \cdot \mathbf{k}_j^c}{\sqrt{d_k}}$$

$$\alpha_{i,j}^c = \text{softmax}(\text{score}^c(\mathbf{x}_i, \mathbf{x}_j))\ \forall j \le i$$

$$\mathbf{z}_i^c = \sum_{j \le i} \alpha_{ij}^c \mathbf{v}_j^c$$

$$\mathbf{a}_i = (\mathbf{z}^1 \oplus \mathbf{z}^2 \ldots \oplus \mathbf{z}^A)\mathbf{W}^O$$

Multi-head attention

Self-attention
multi-head attention

The intuition behind this is that different heads might be attending to the context for different purposes:

- different linguistic relationships
- particular kinds of patterns

In multi-head attention we have A separate attention heads, each with their own set of key, query, value matrices. The dimensions from the single head still hold.

In the original Transformers paper they were:

$$d_k, d_v = 64,\ A = 8,\ d = 512$$

$$\mathbf{x}_i \in \mathbb{R}^d;\ \ \mathbf{W}^{Q_c}, \mathbf{W}^{K_c} \in \mathbb{R}^{d \times d_k};\ \ \mathbf{W}^{V_c} \in \mathbb{R}^{d \times d_v}$$

$$\mathbf{q}_i^c = \mathbf{x}_i \mathbf{W}^{Q_c};\ \ \mathbf{k}_j^c = \mathbf{x}_j \mathbf{W}^{K_c};\ \ \mathbf{v}_j^c = \mathbf{x}_j \mathbf{W}^{V_c}; \quad \forall c\ 1 \leq c \leq A$$

$$\text{score}^c(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i^c \cdot \mathbf{k}_j^c}{\sqrt{d_k}}$$

$$\alpha_{i,j}^c = \text{softmax}(\text{score}^c(\mathbf{x}_i, \mathbf{x}_j))\ \forall j \leq i$$

$$\mathbf{z}_i^c = \sum_{j \leq i} \alpha_{ij}^c \mathbf{v}_j^c$$

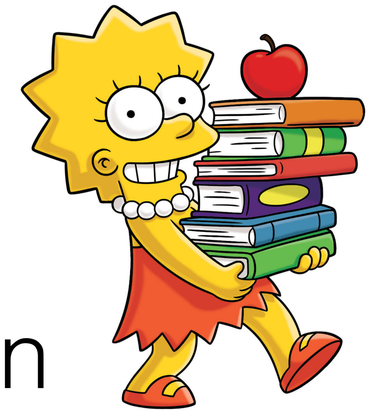
$$\mathbf{a}_i = (\mathbf{z}^1 \oplus \mathbf{z}^2 \ldots \oplus \mathbf{z}^A)\mathbf{W}^O$$

$\mathbf{z}^1, \ldots, \mathbf{z}^A$ are concatenated and projected to d as before

Let’s borrow a visualization from Strubell et al.

This is a simplified visualization, the relation between input d and hidden dimensions d_v, d_k are not accurate.

LISA



Linguistically-Informed Self-Attention for Semantic Role Labeling

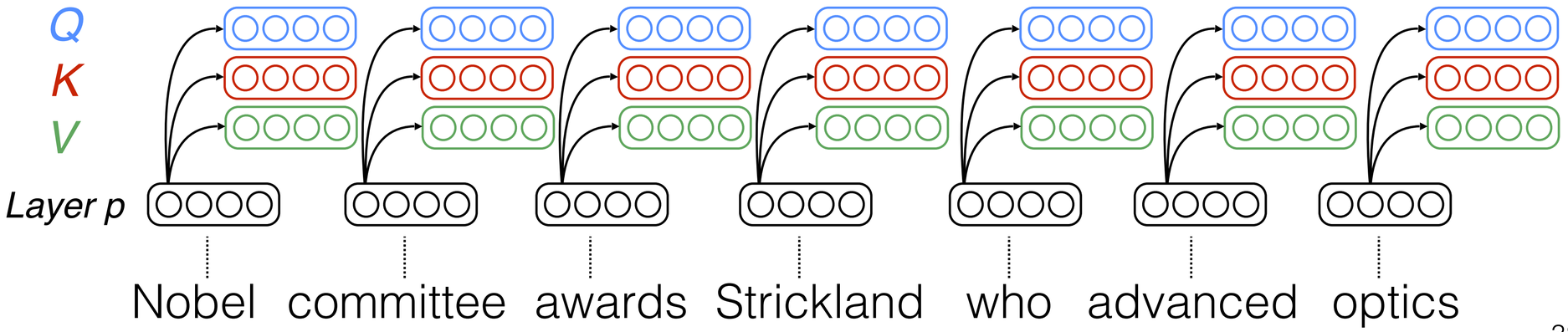
Emma Strubell[1] Patrick Verga[1] Daniel Andor[2] David Weiss[2] Andrew McCallum[1]

[1] UMassAmherst College of Information and Computer Sciences

[2] Google AI

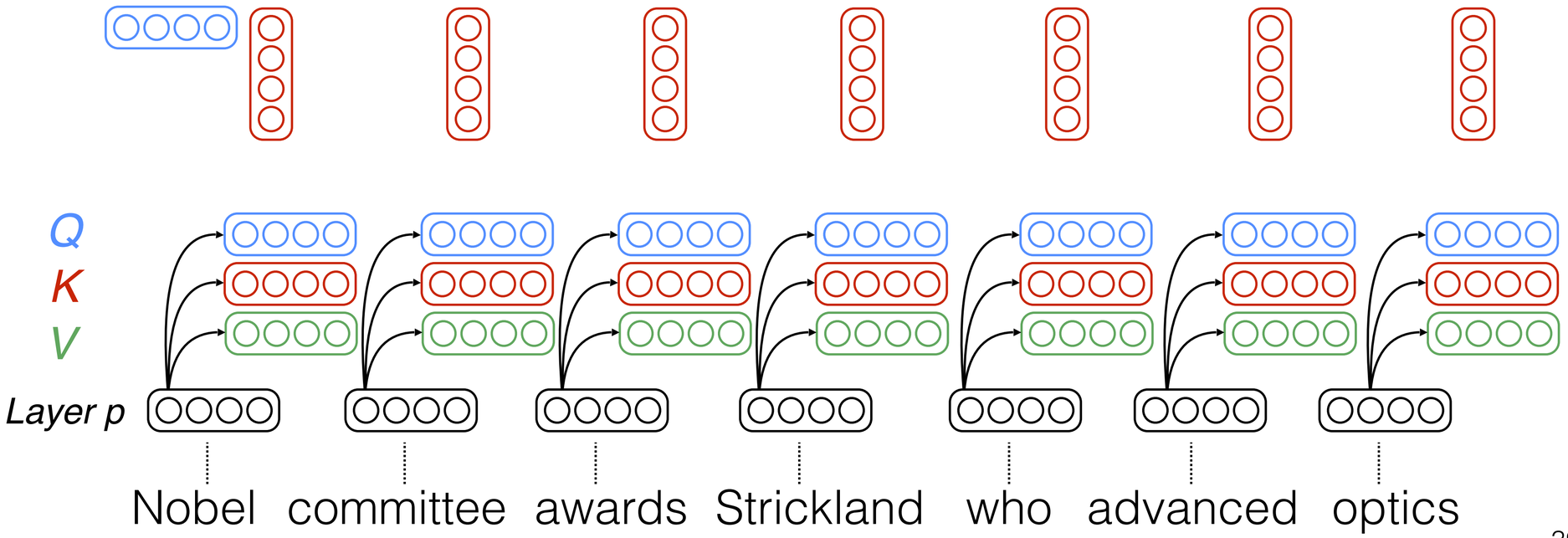
[Vaswani et al. 2017]

Self-attention



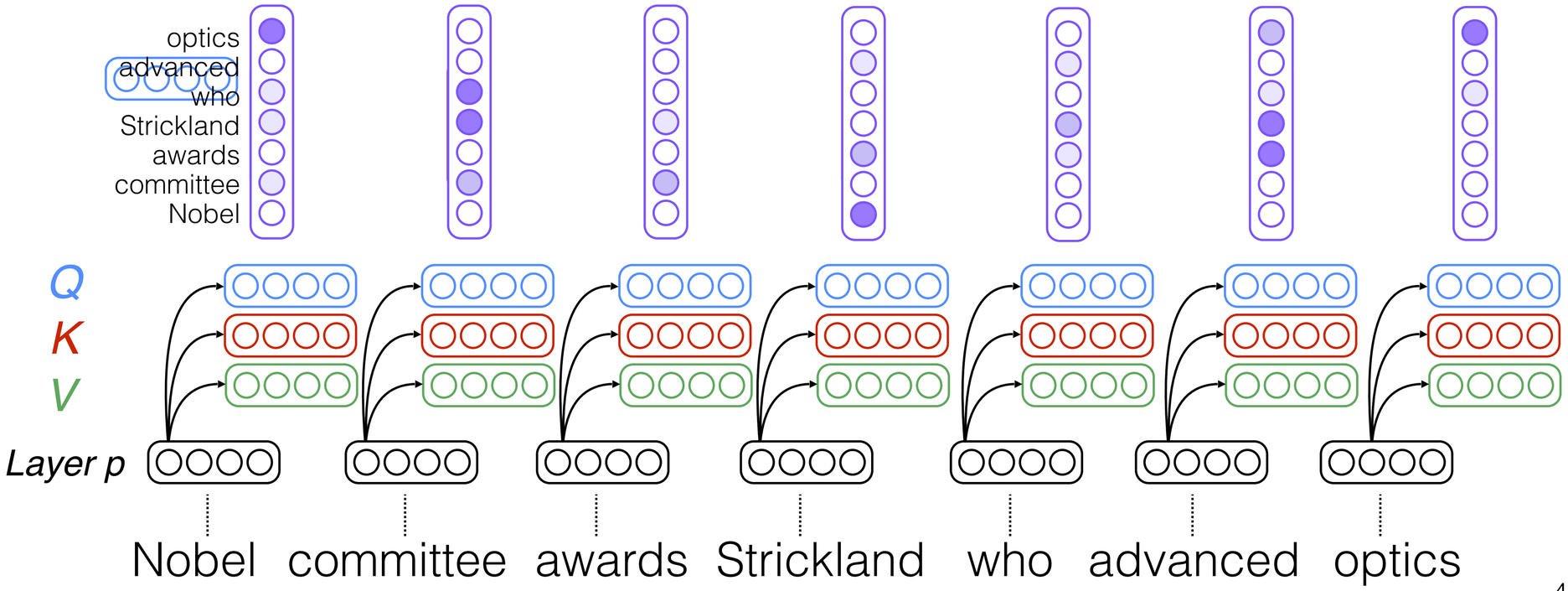
[Vaswani et al. 2017]

Self-attention



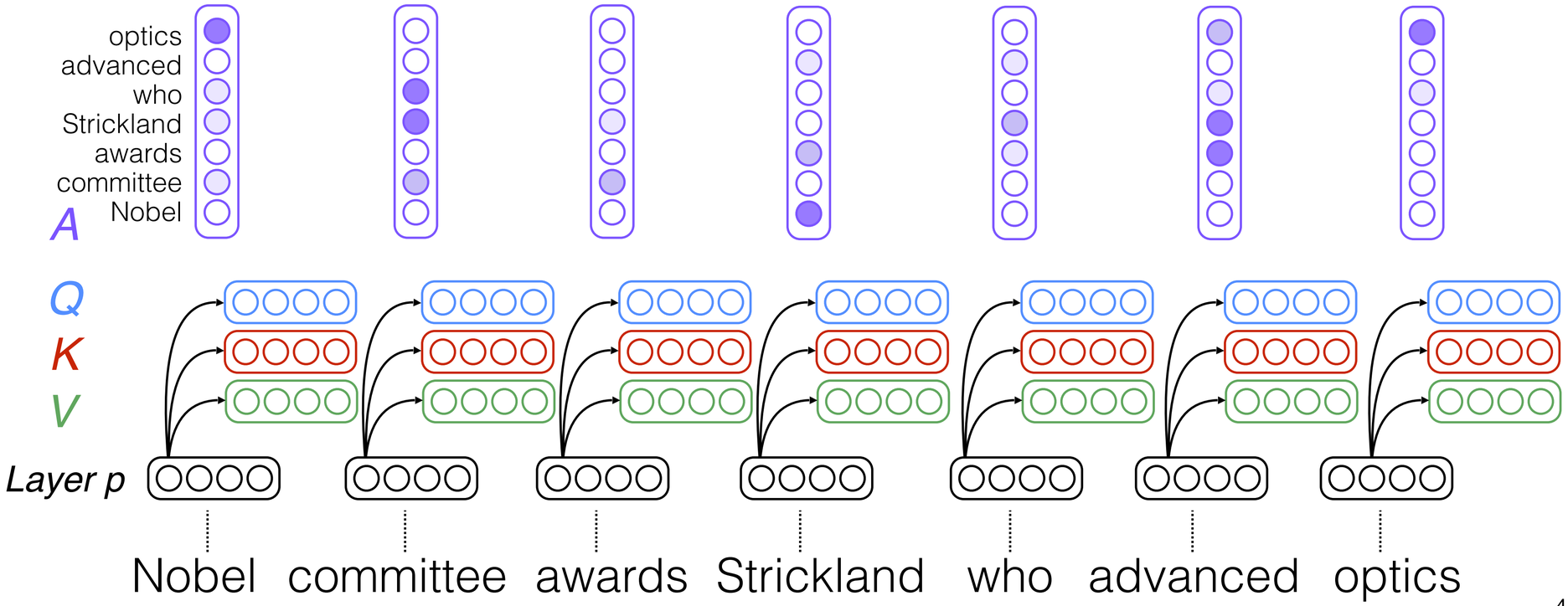
[Vaswani et al. 2017]

Self-attention



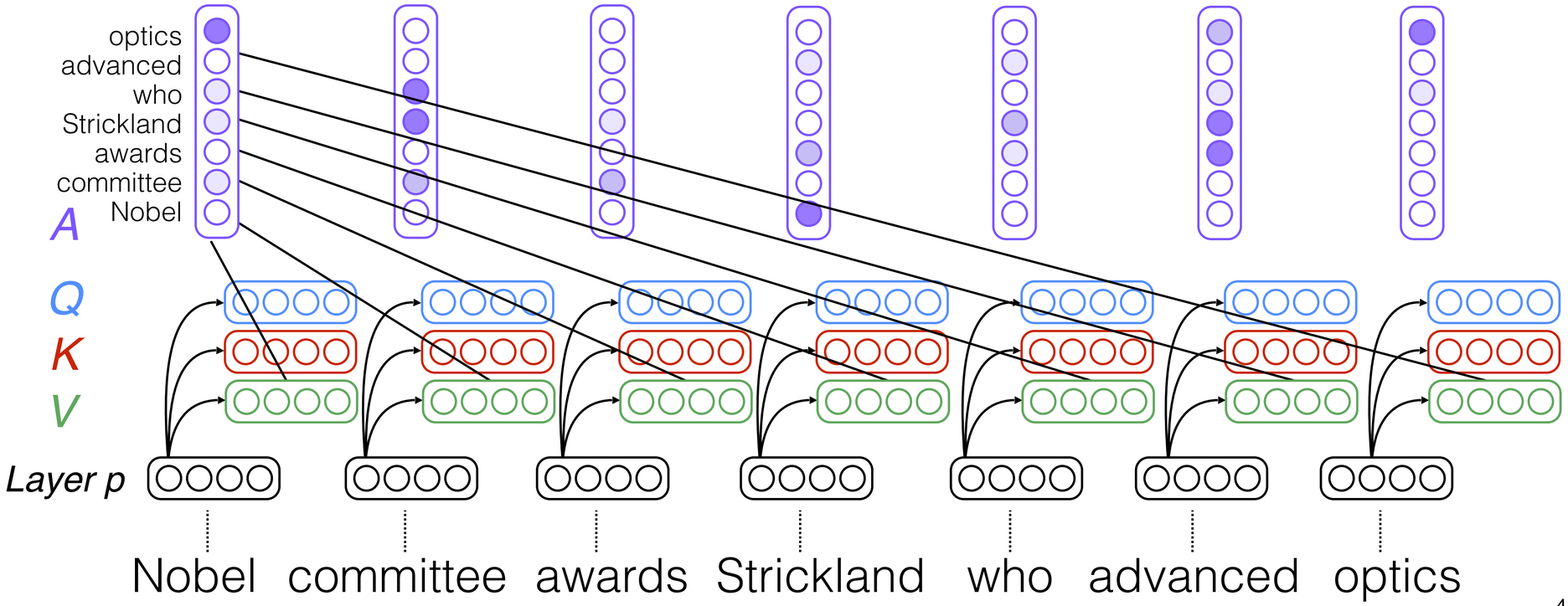
[Vaswani et al. 2017]

Self-attention



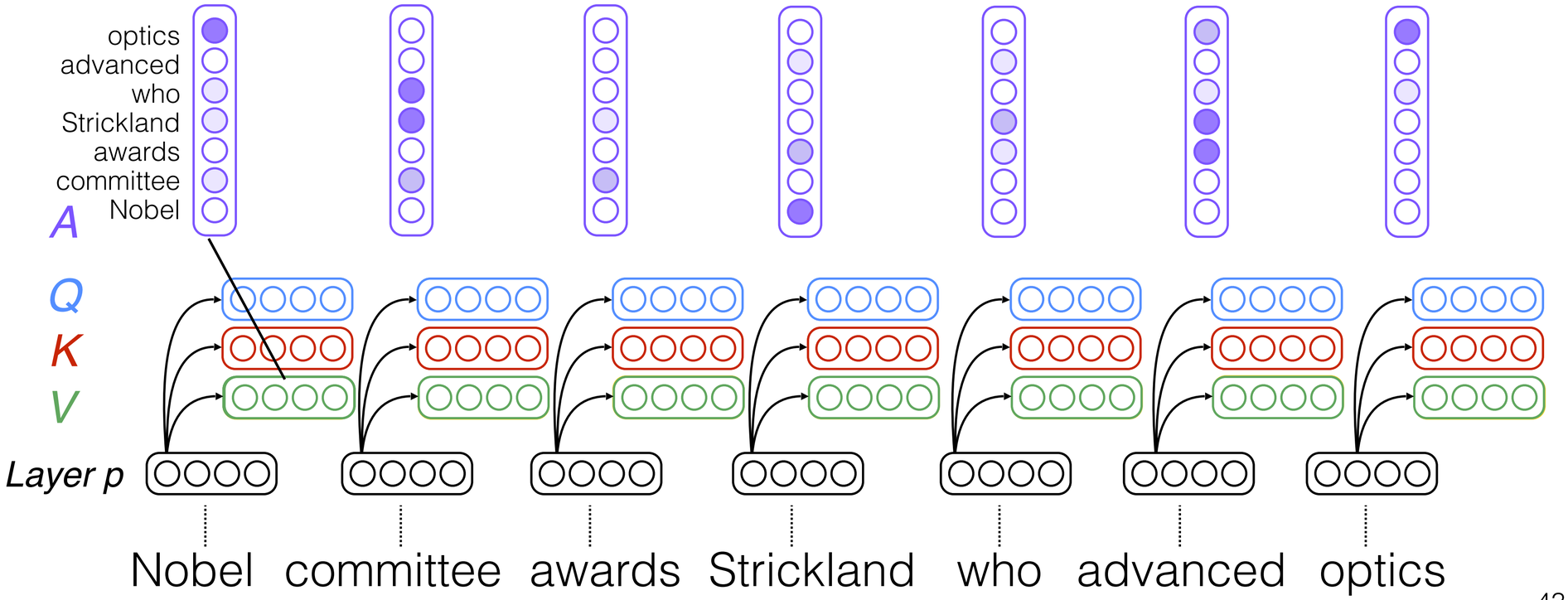
[Vaswani et al. 2017]

Self-attention



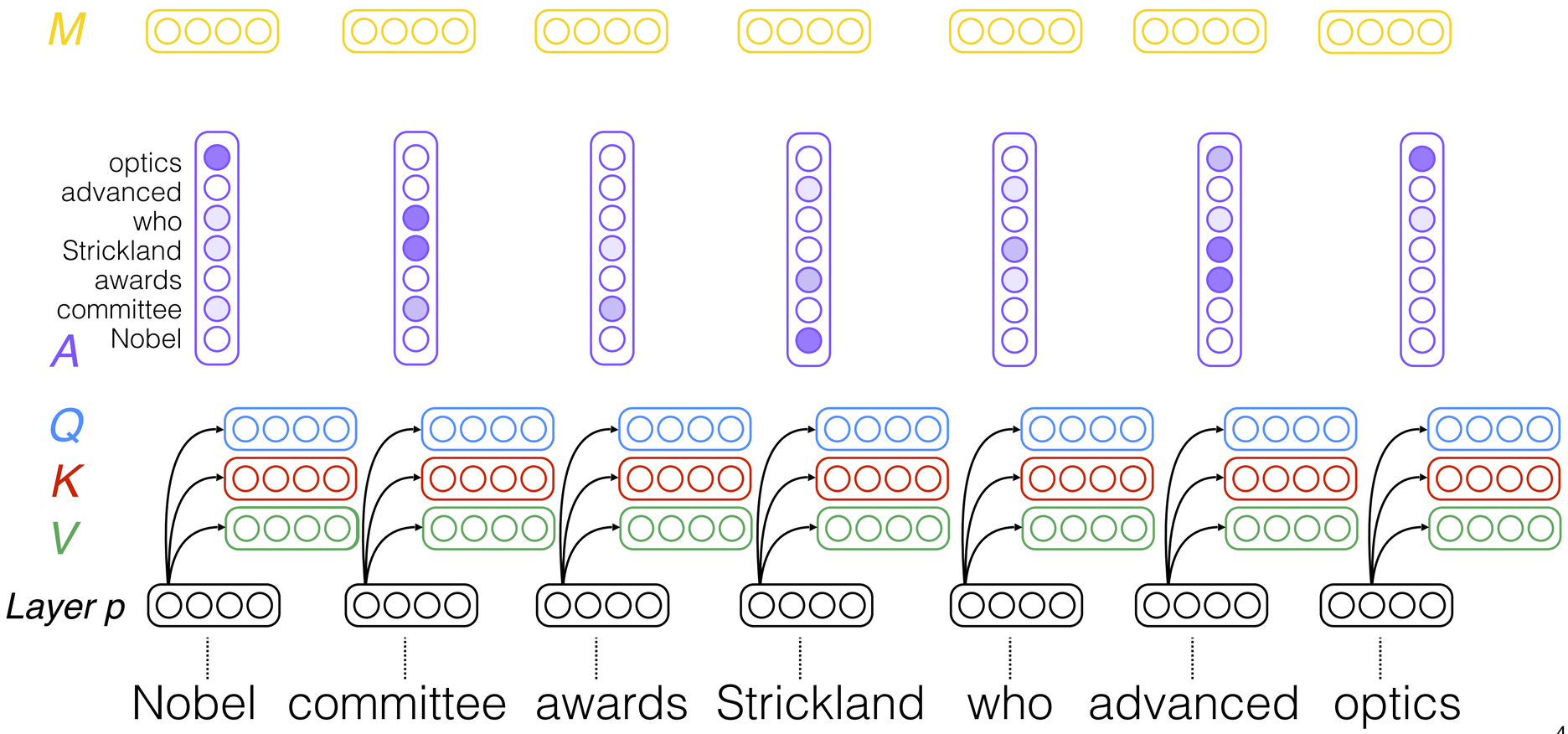
[Vaswani et al. 2017]

Self-attention



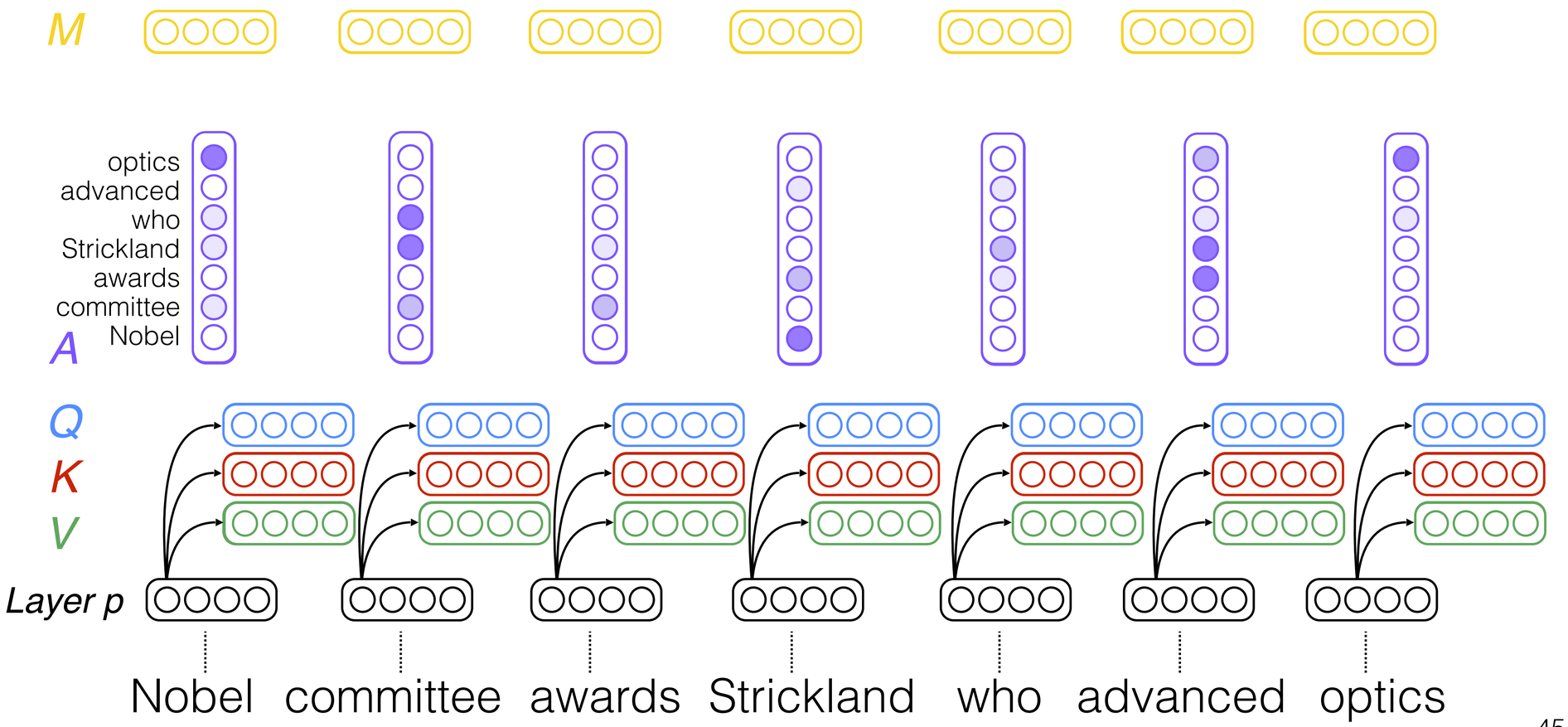
[Vaswani et al. 2017]

Self-attention



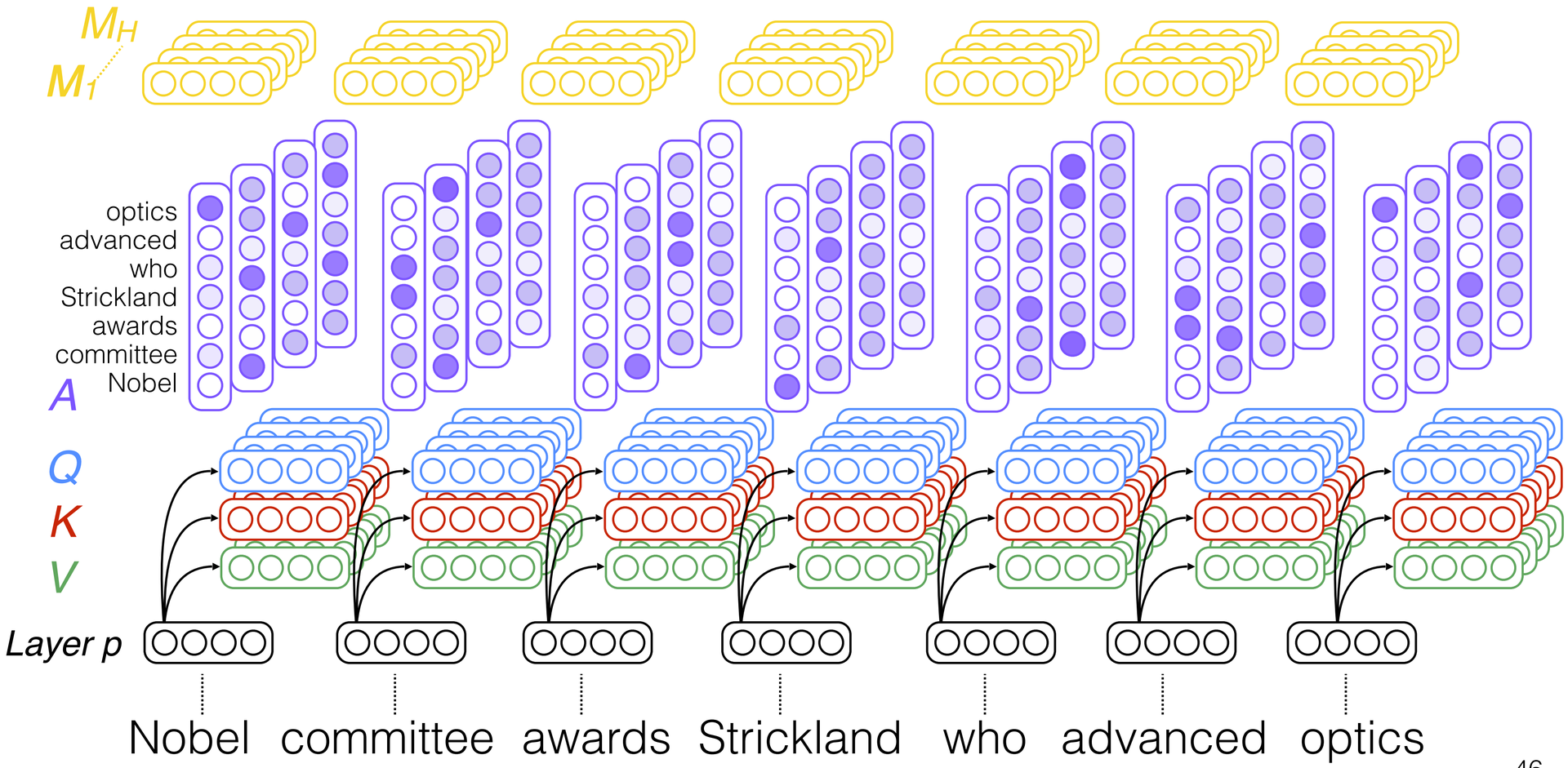
[Vaswani et al. 2017]

Self-attention



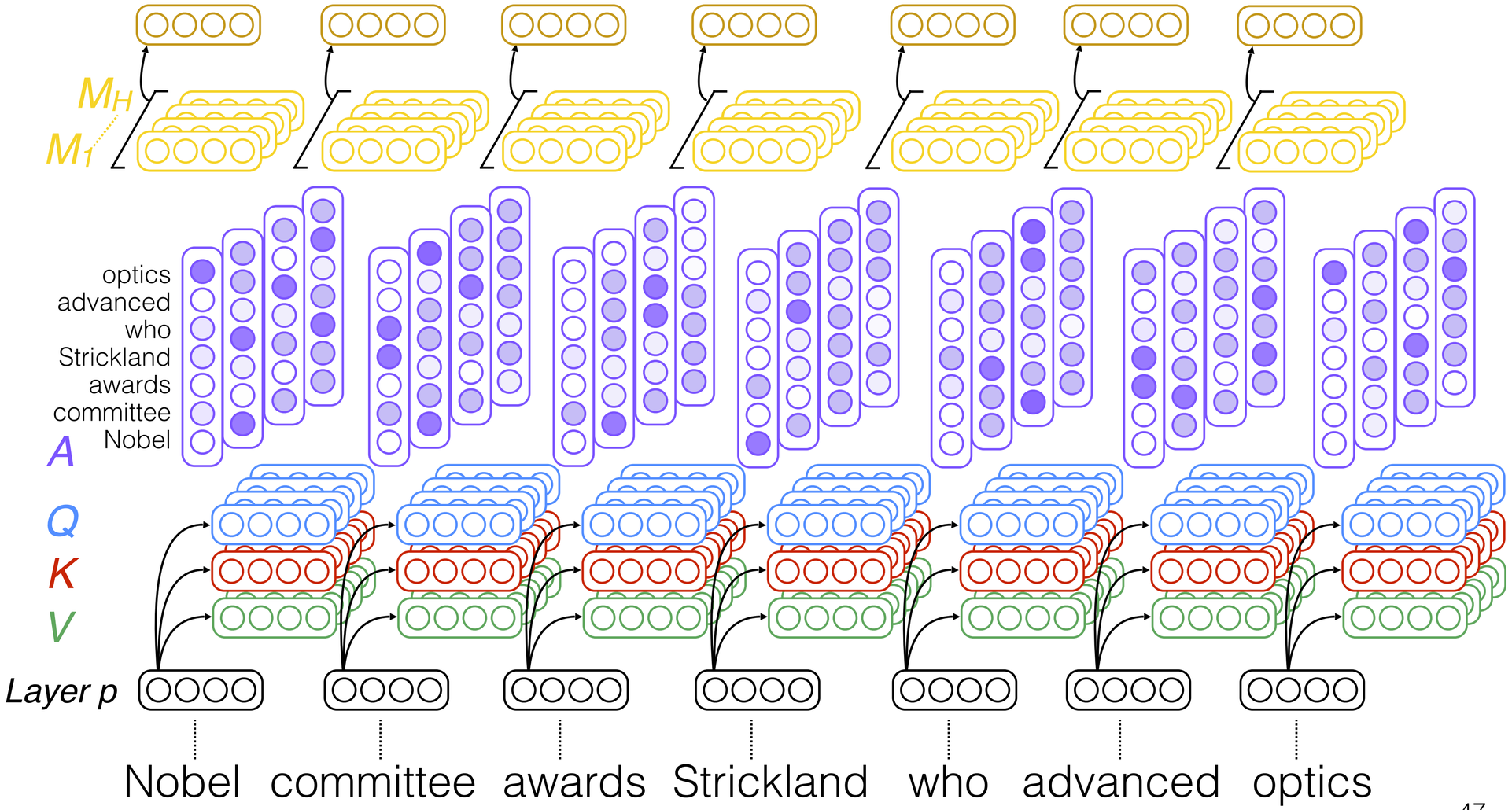
[Vaswani et al. 2017]

Multi-head self-attention



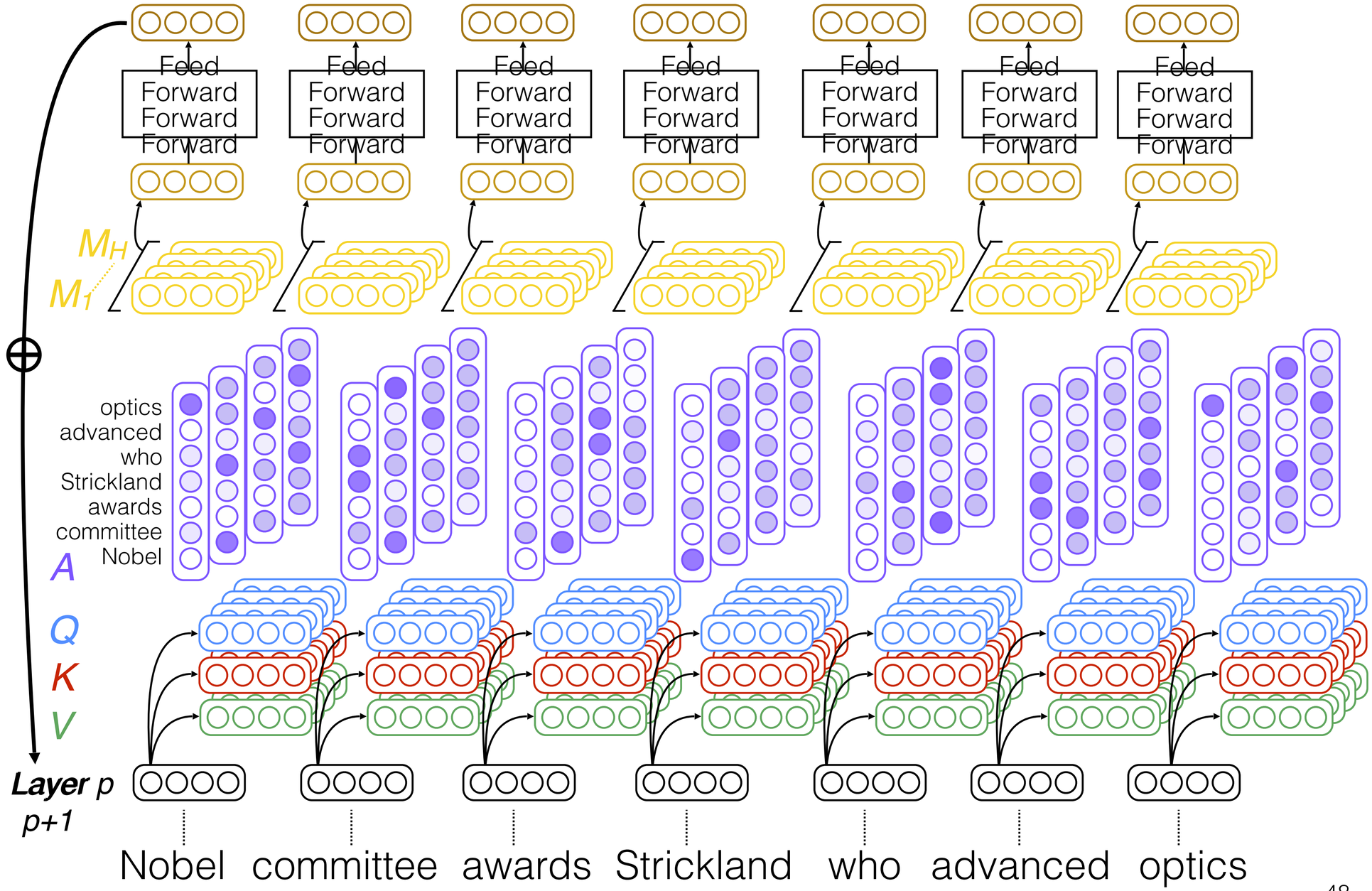
[Vaswani et al. 2017]

Multi-head self-attention



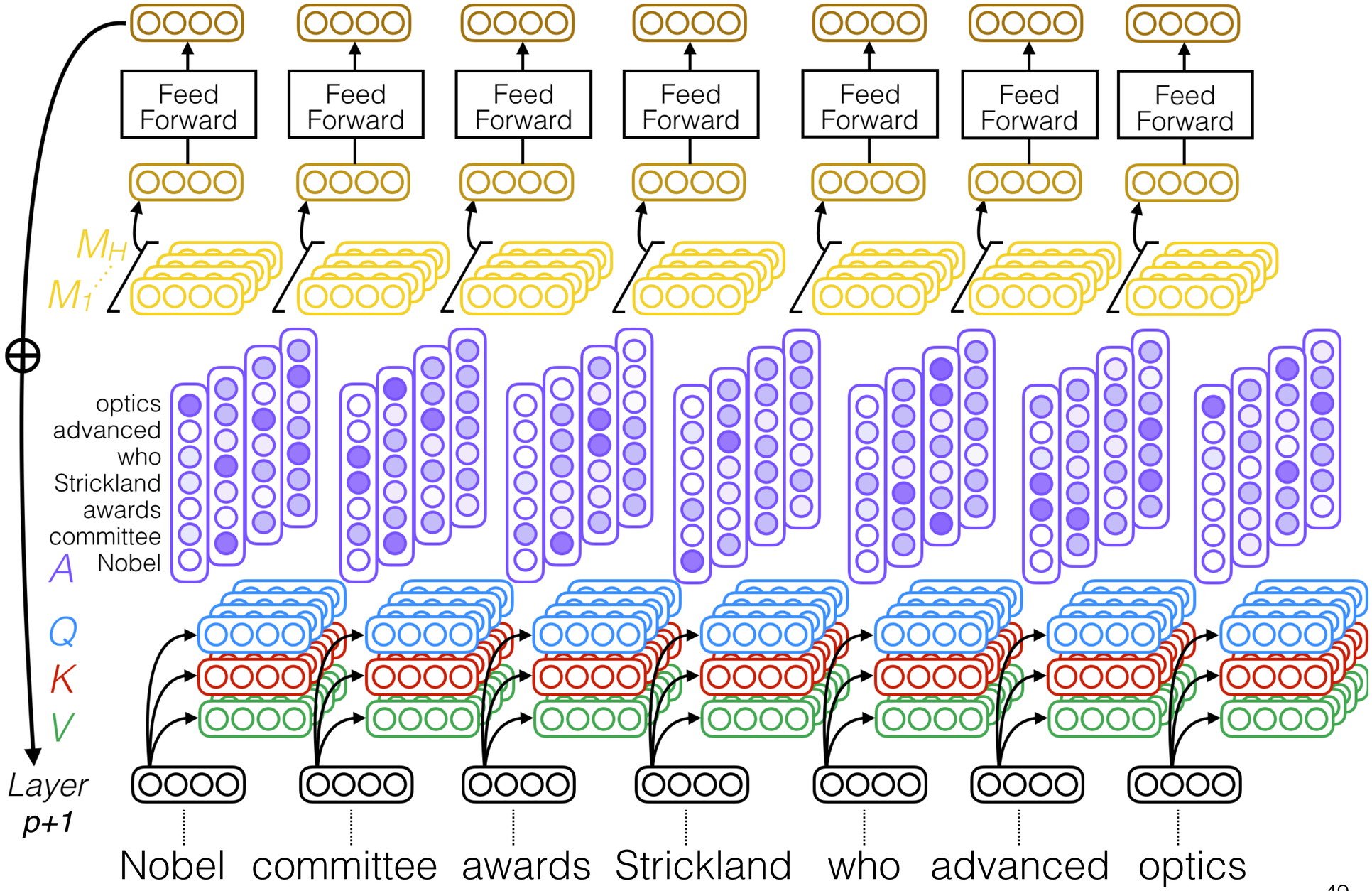
[Vaswani et al. 2017]

Multi-head self-attention



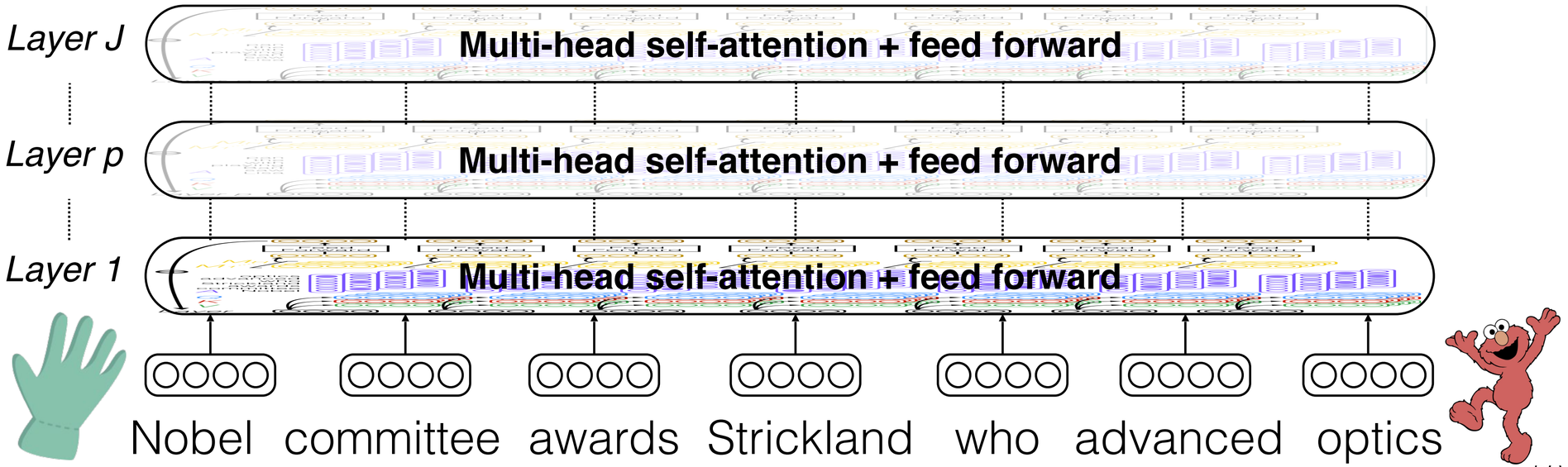
[Vaswani et al. 2017]

Multi-head self-attention



[Vaswani et al. 2017]

Multi-head self-attention



Residuals, Layer Norm, FFN

The Transformer Architecture

Encoder-Decoder

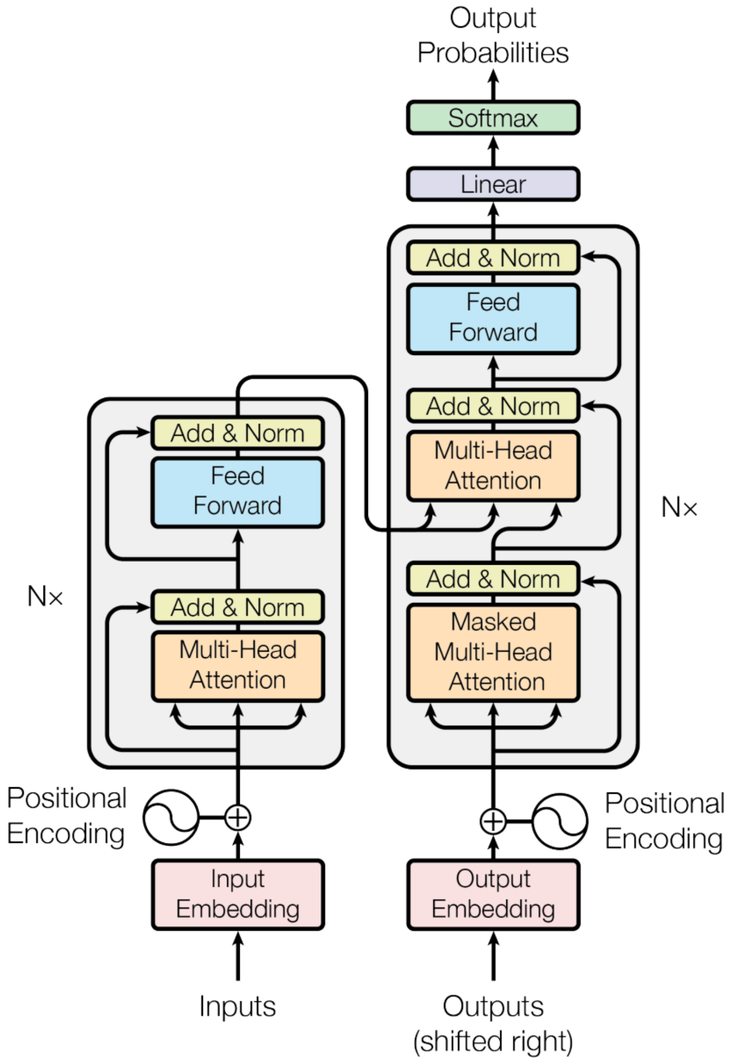


Figure 1: The Transformer - model architecture.

The Transformer Architecture

~~Encoder~~-Decoder

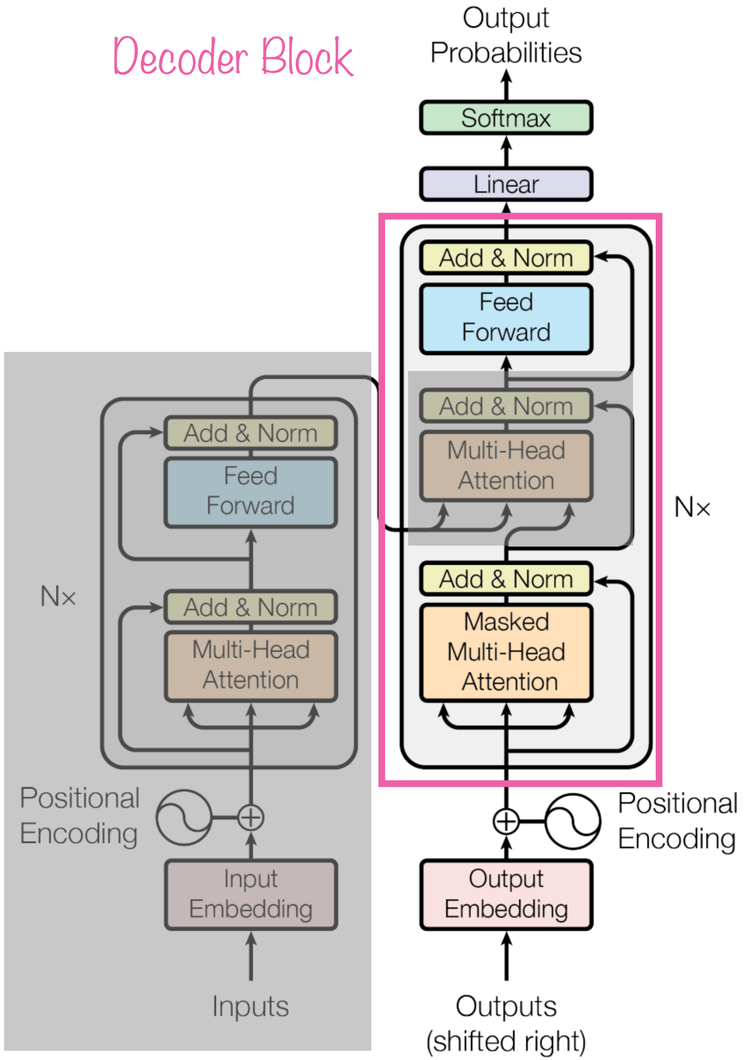


Figure 1: The Transformer - model architecture.

- We are focusing on decoder-only models
  - We don't need the encoder
  - We only use masked multi-head attention (another way of saying we only consider past tokens)

The Transformer Architecture

Decoder-only

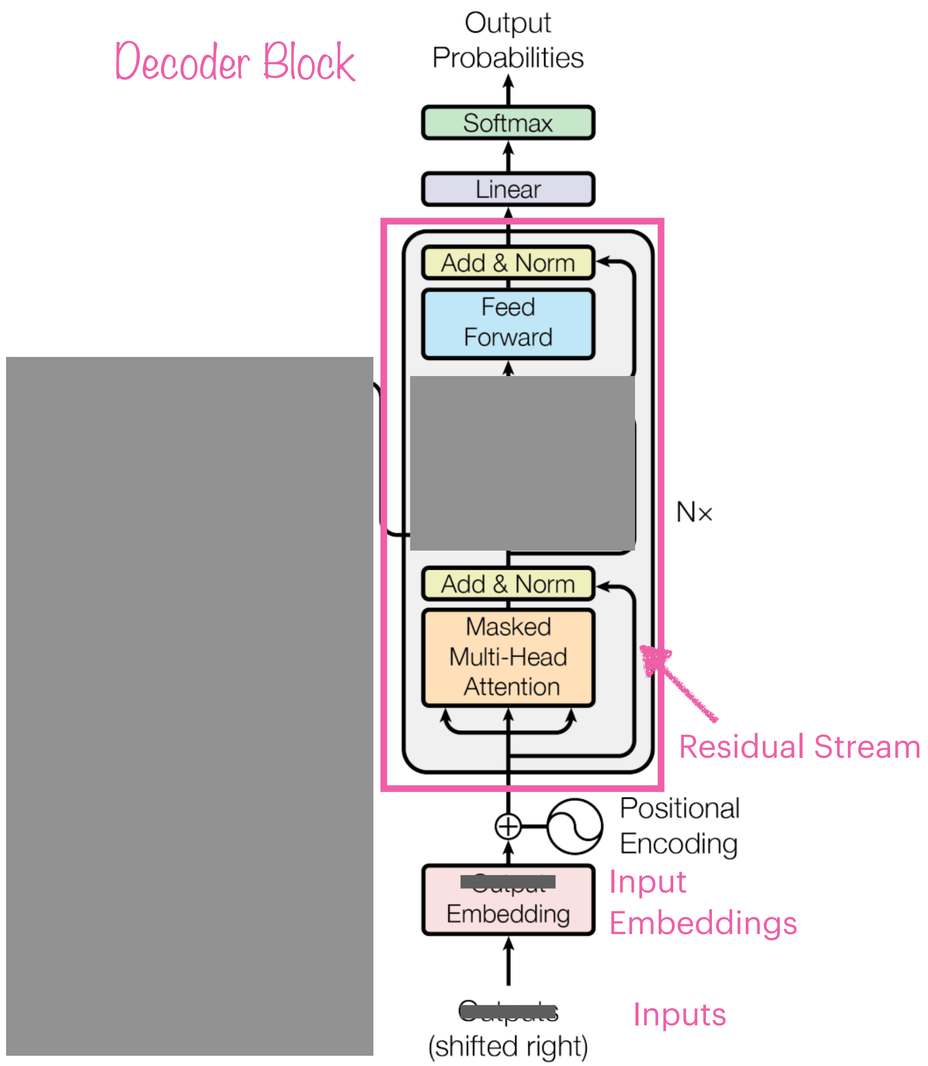


Figure 1: The Transformer - model architecture.

- We are focusing on decoder-only models
  - We don't need the encoder
  - We only use masked multi-head attention (another way of saying we only consider past tokens)
- A decoder contains several components
  - Residual Stream
  - self-attention, here "Masked Multi-Head Attention"
  - Layer Norm, here "Norm"
  - Feed Forward Layer

The Transformer Architecture

Decoder-only

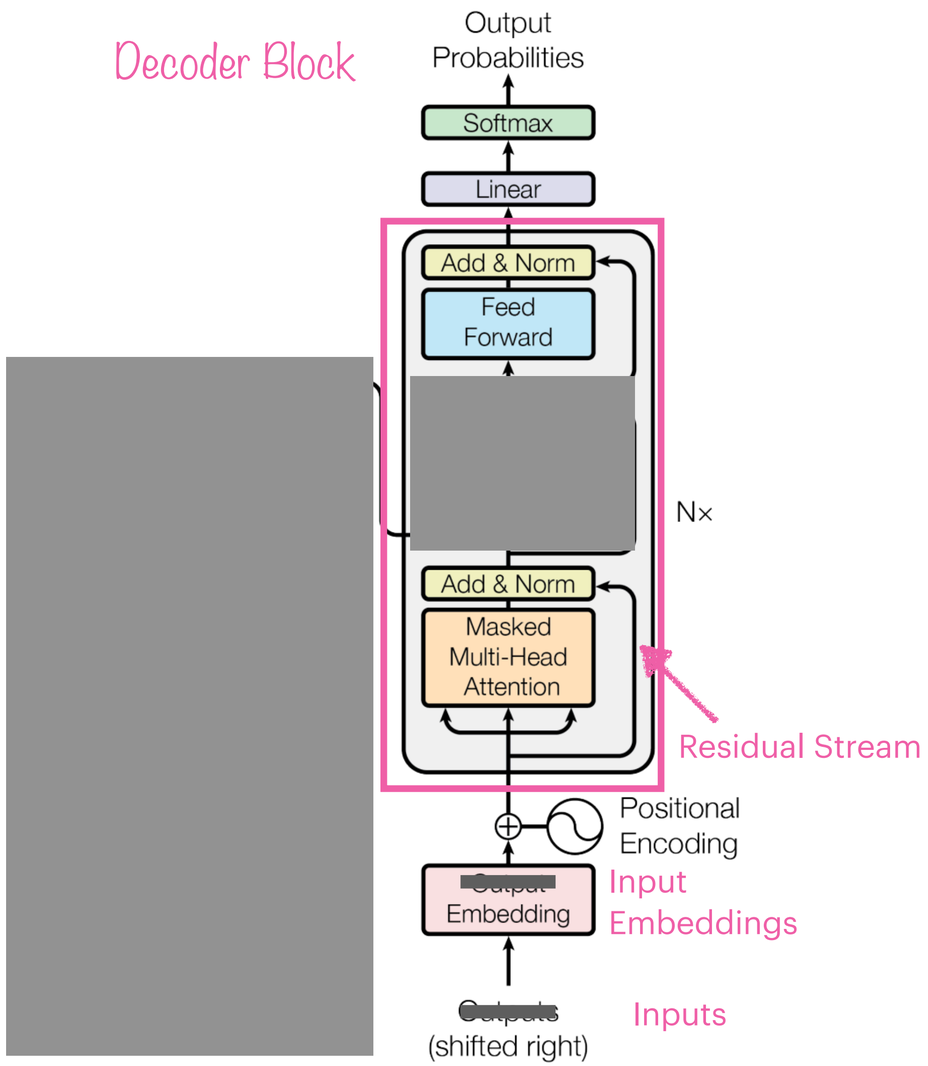


Figure 1: The Transformer - model architecture.

- A decoder contains several components
  - Residual Stream
  - self-attention, here “Masked Multi-Head Attention”
  - Layer Norm, here “Norm”
  - Feed Forward Layer

Residual Stream

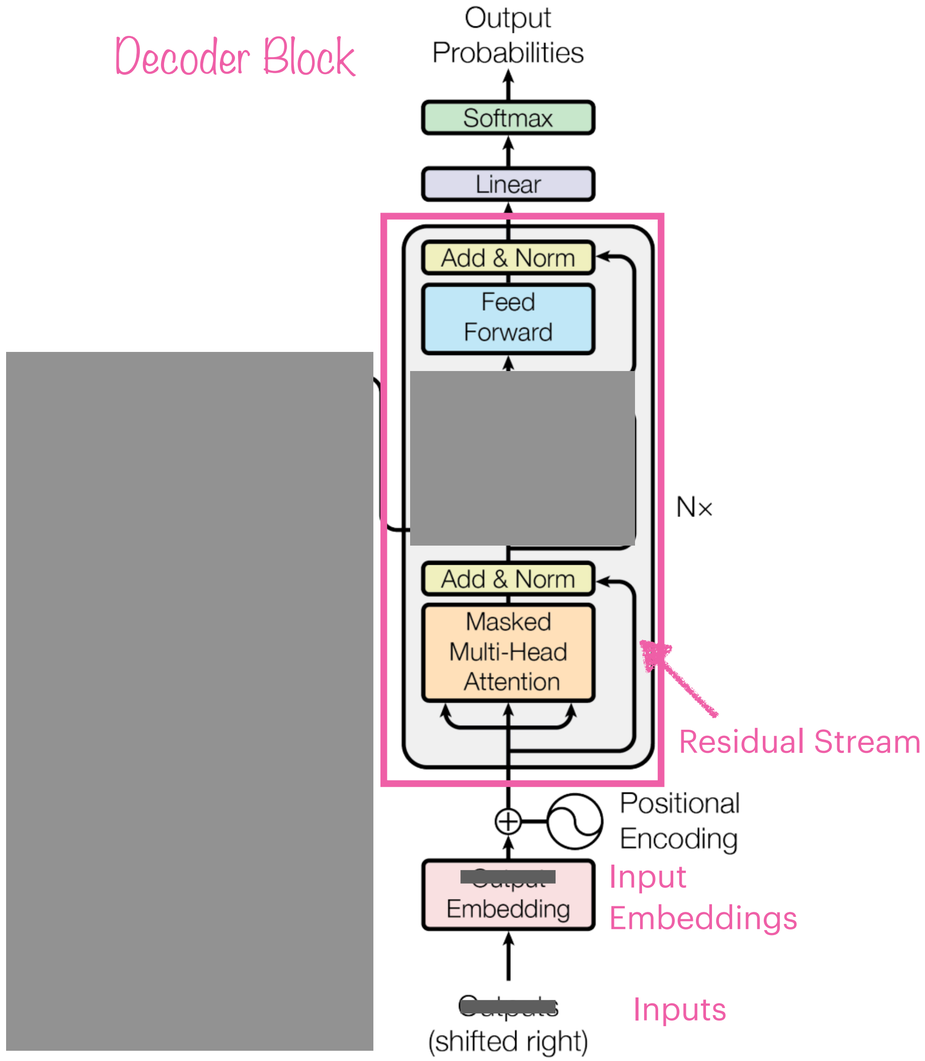


Figure 1: The Transformer - model architecture.

- The residual stream takes the output of a layer and adds it to the input of a layer
- For example for the self-attention layer:
  - we take the original embedding x_i
  - and update it with the contextualized embedding a_i

    ➡️ $\tilde{x}_i = x_i + a_i$
- It acts as a shared memory between layers, every layer can access information from previous layers
- They are also called “skip connections”

The Transformer Architecture

Decoder-only

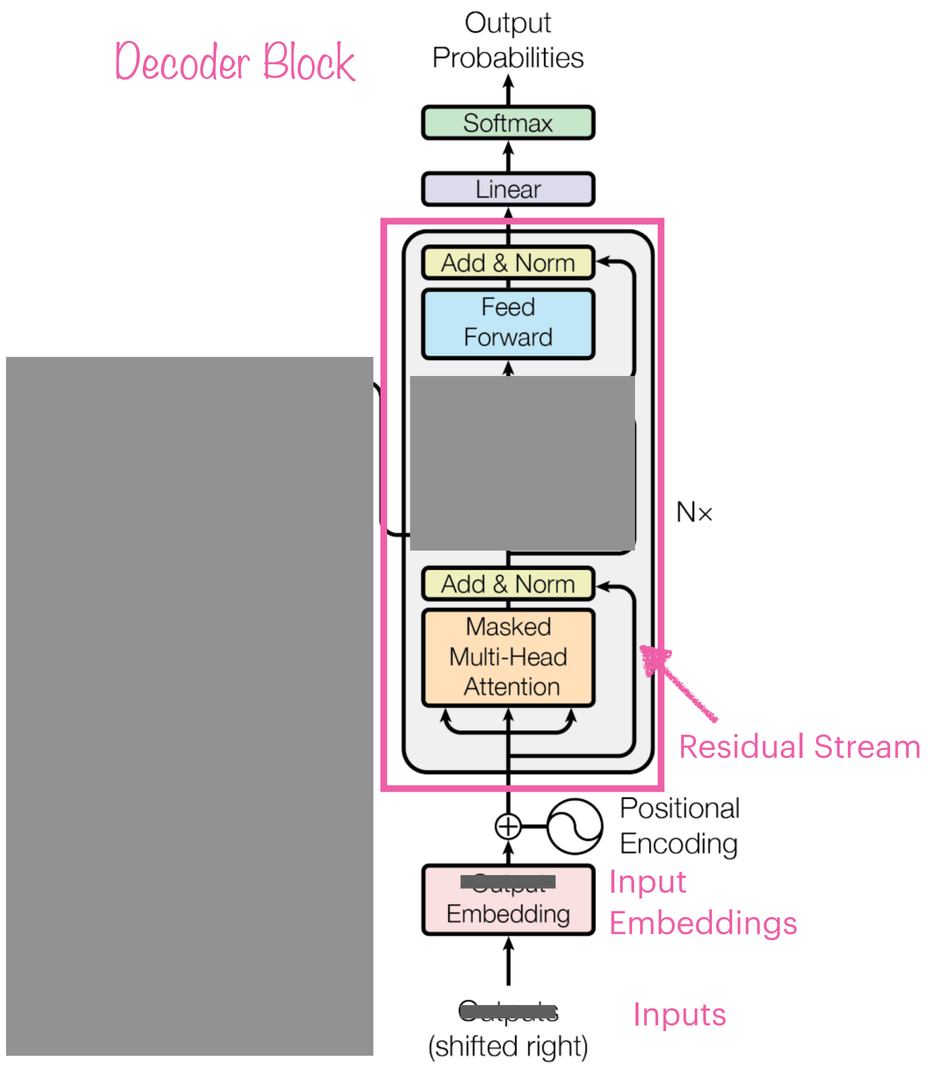


Figure 1: The Transformer - model architecture.

- A decoder contains several components
  - Residual Stream ✅
  - self-attention, here “Masked Multi-Head Attention” ✅
  - Layer Norm, here “Norm”
  - Feed Forward Layer

Layer Norm

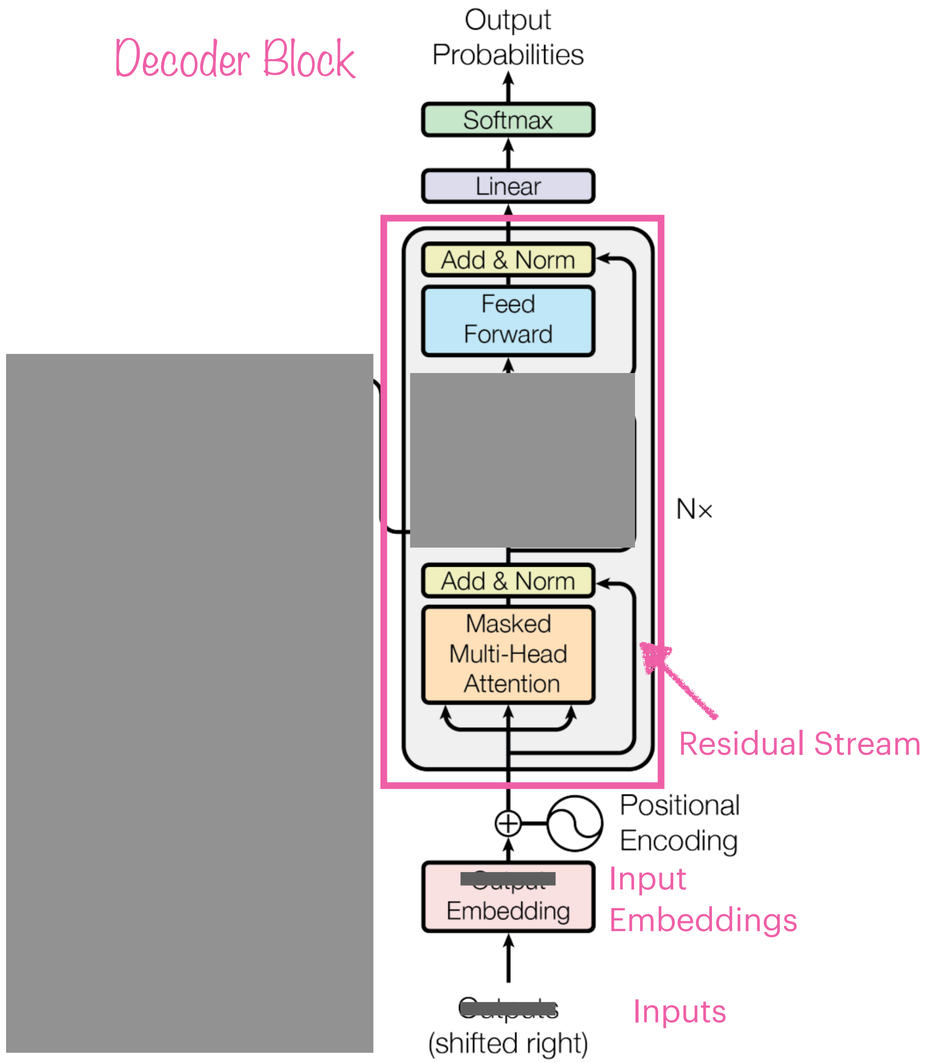


Figure 1: The Transformer - model architecture.

- Layer norm, short for "layer normalization", although we only normalize individual vectors not the entire layer
- at two points in the Transformer, we normalize the embedding vectors x_i:
  - after the attention block
  - after the feedforward layer

$$\mu_i = \frac{1}{d}\sum_{j=1}^{d} x_{ij}$$

$$\sigma_i = \sqrt{\frac{1}{d}\sum_{j=1}^{d}(x_{ij} - \mu_i)^2}$$

$$\hat{\mathbf{x}}_i = \frac{\mathbf{x}_i - \mu_i}{\sigma_i}$$

- to improve training performance and keep the values in a certain range that facilitates gradient-based training

The Transformer Architecture

Decoder-only

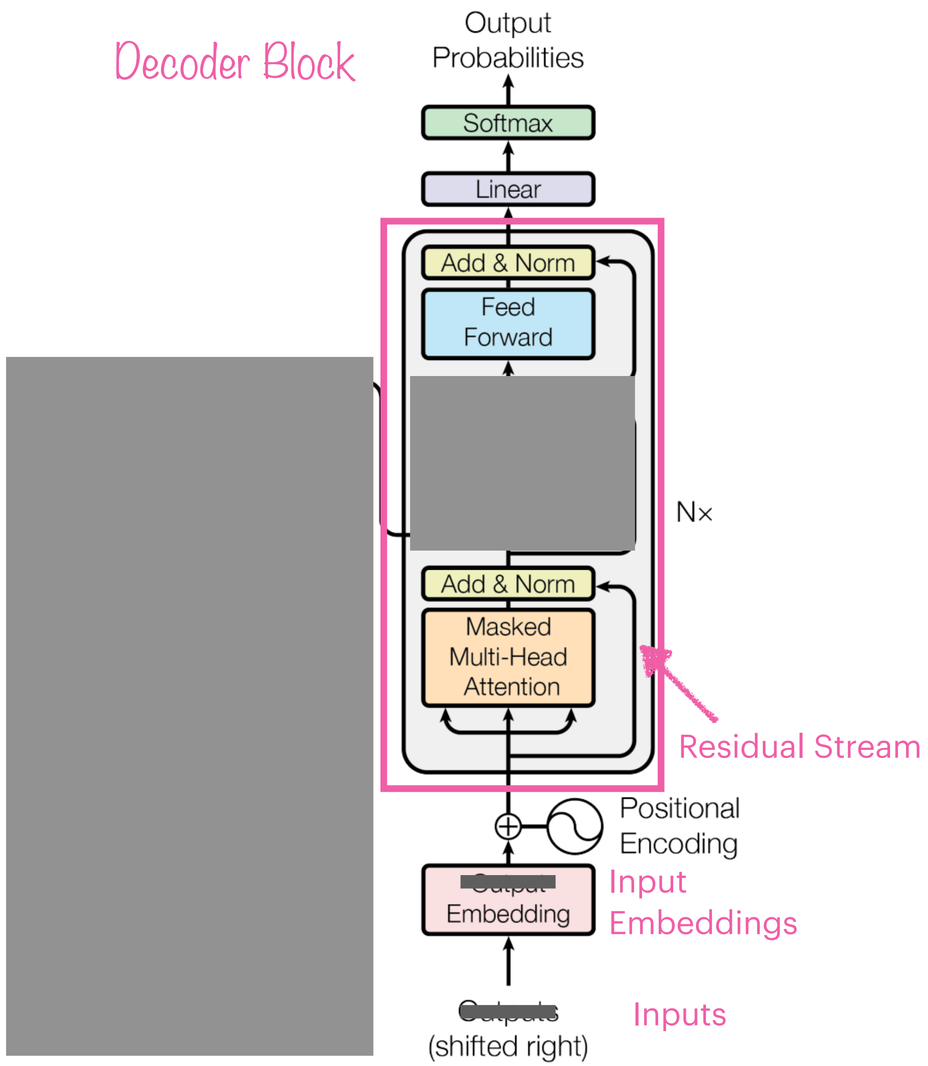


Figure 1: The Transformer - model architecture.

- A decoder contains several components
  - Residual Stream ✅
  - self-attention, here “Masked Multi-Head Attention” ✅
  - Layer Norm, here “Norm” ✅
  - Feed Forward Layer

Feed forward layer

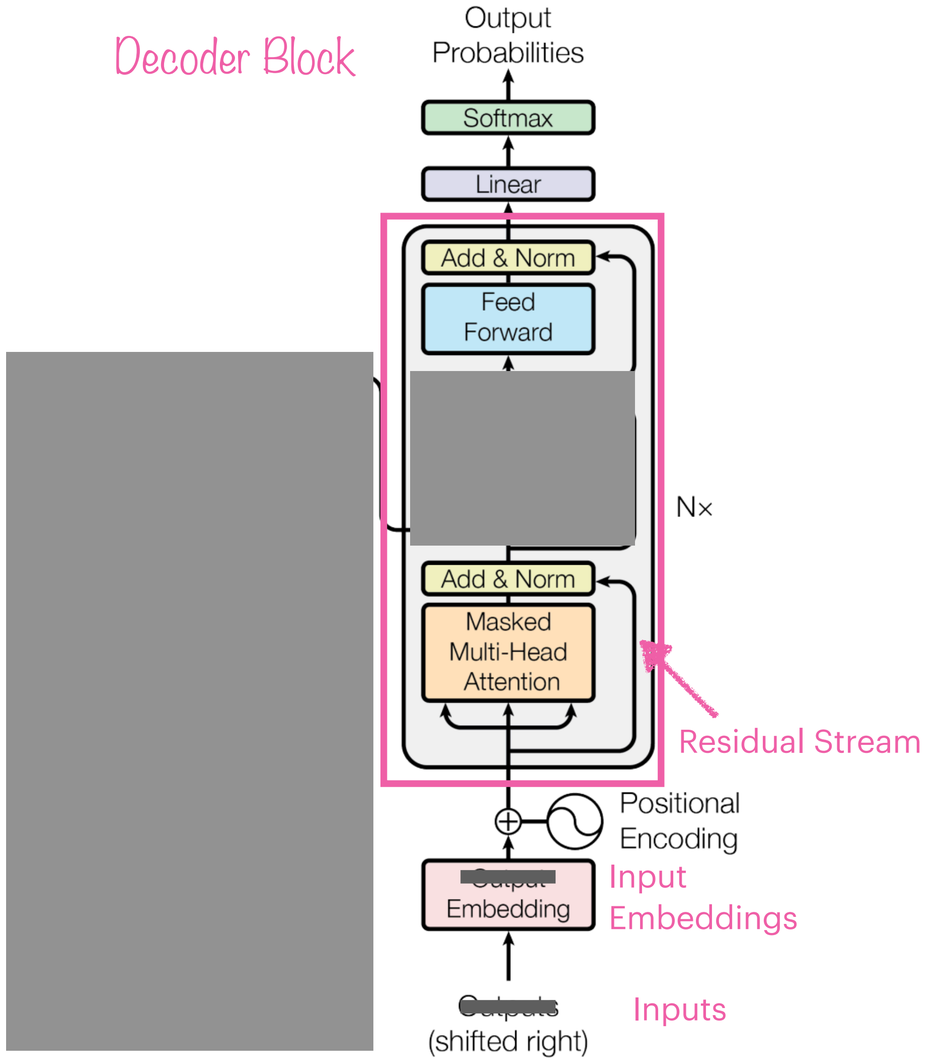


Figure 1: The Transformer - model architecture.

- The feed forward layer is applied after the attention block
- It collects what is learned in the attention block and processes it (think of it as a conclusion)
- It is a fully-connected 2-layer network, i.e., one hidden layer and two weight matrices
- It introduces non-linear transformations, thus enabling the model to capture complex relationships between tokens
- By operating on each word independently, the network can capture local patterns and generate higher-level representations of the sequence.

$$\text{FFN}(\mathbf{x}_i) = \text{ReLU}(\mathbf{x}_i\mathbf{W}_1 + b_1)\mathbf{W}_2 + b_2$$

Outside of the Decoder

Positional Embedding

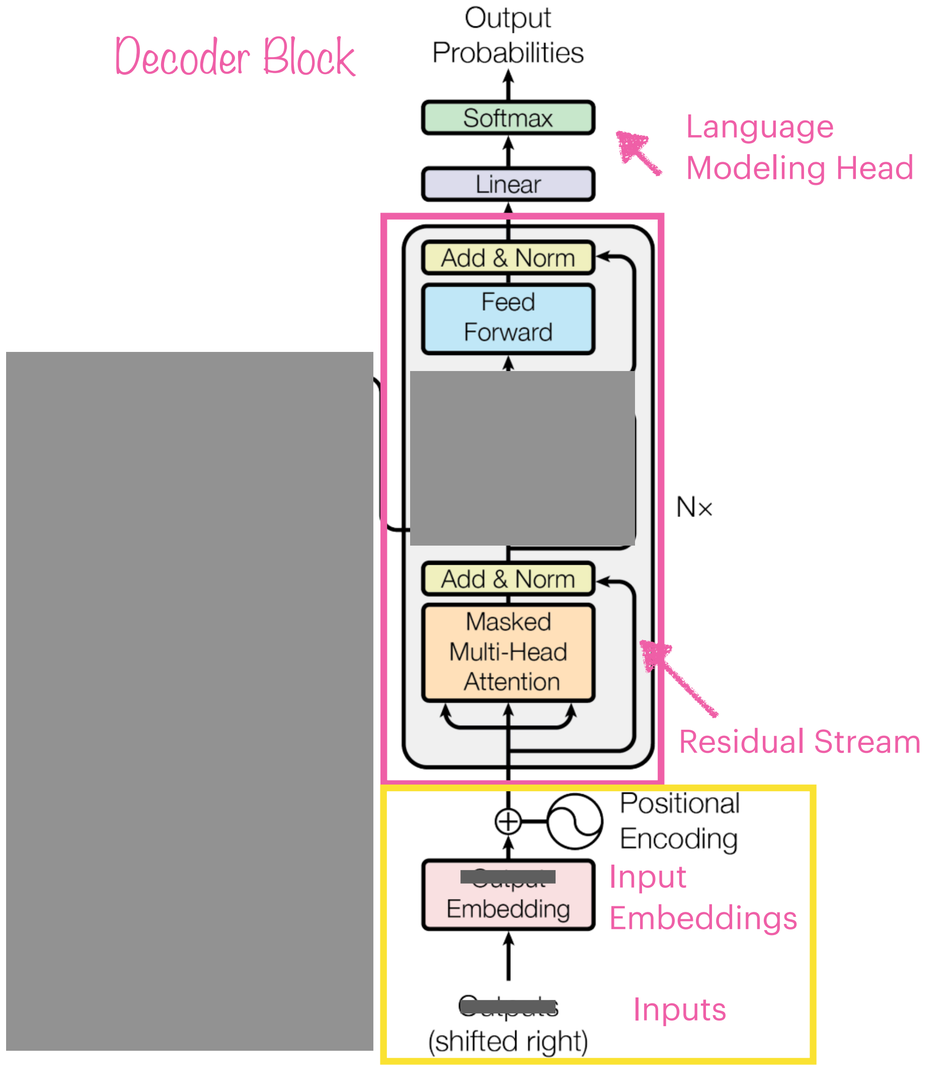


Figure 1: The Transformer - model architecture.

- The input to the Transformer is a sequence of tokens
- Tokens have to be transformed into numbers to continue computations → embeddings
- Those input embeddings are not position-dependent, so we add **positional embeddings**
- The following two sentences contain the same words at different positions:

Are we going to the movies
We are going to the movies

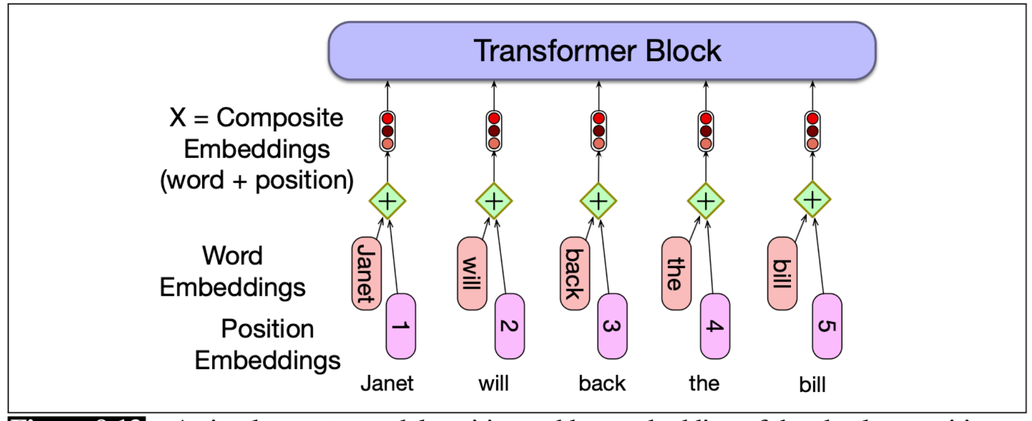


Figure 9.13 A simple way to model position: add an embedding of the absolute position to the token embedding to produce a new embedding of the same dimensionality.

Language Modeling Head

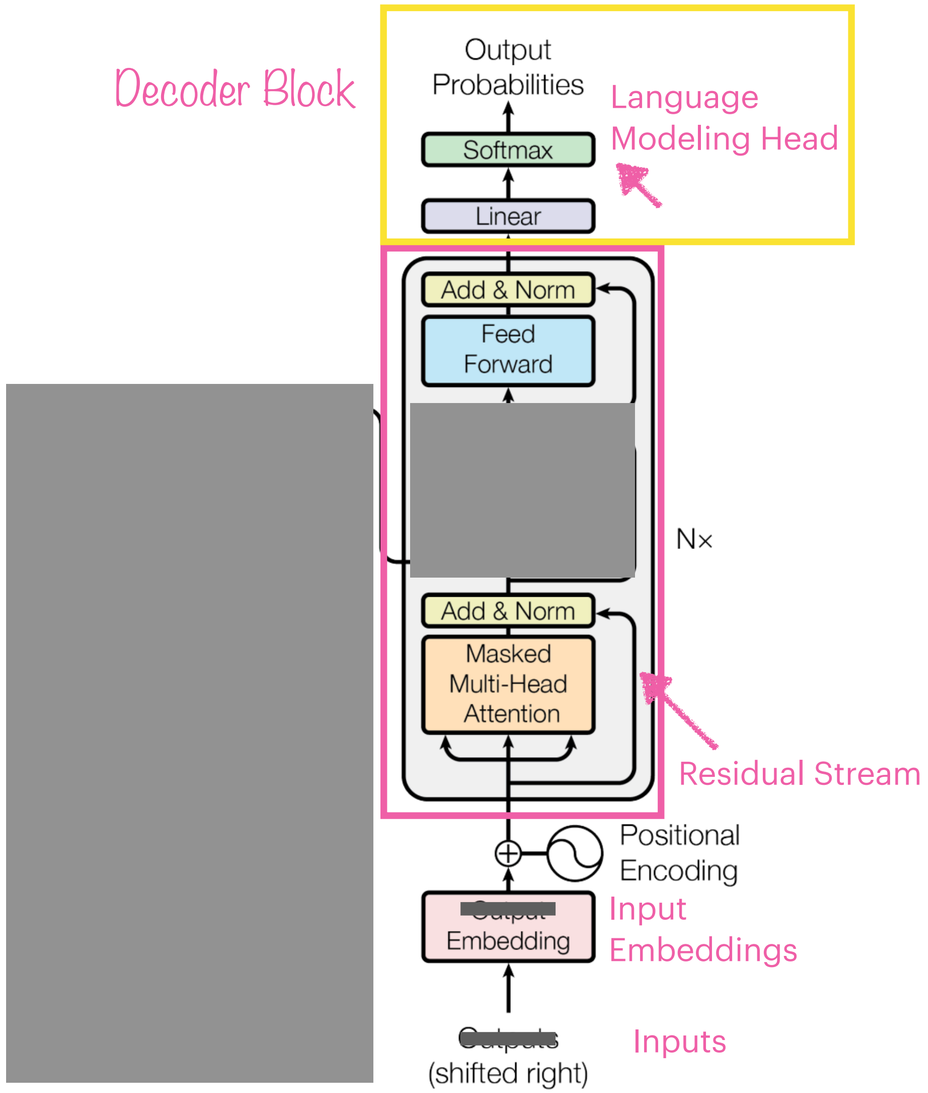
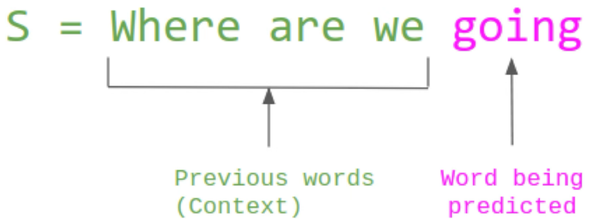


Figure 1: The Transformer - model architecture.

- The language modeling head helps us to assign word probabilities at the end of the language model
- It takes the output of the final transformer layer from the last token N and uses it to predict the upcoming word at position $N + 1$



$$P(S) = P(\text{Where}) \times P(\text{are} \mid \text{Where}) \times P(\text{we} \mid \text{Where are}) \times P(\text{going} \mid \text{Where are we})$$

Reminder of the main LM task

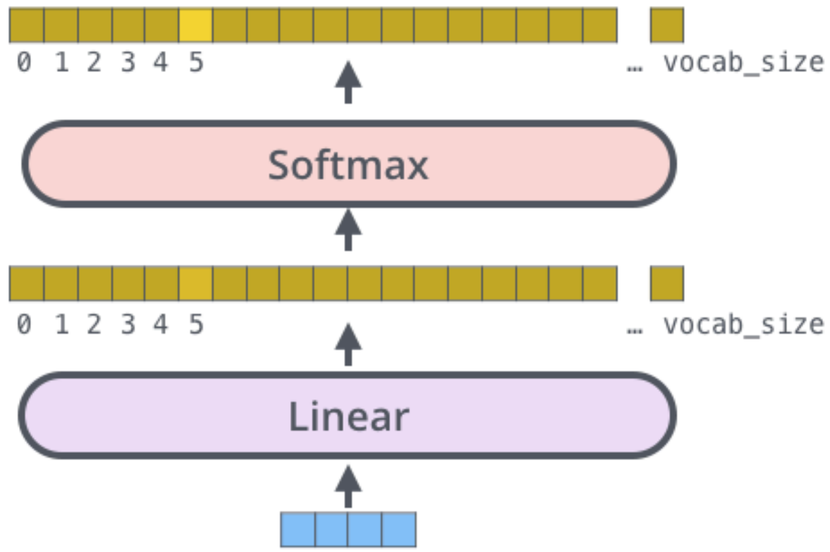
Language Modeling Head

Which word in our vocabulary is associated with this index?

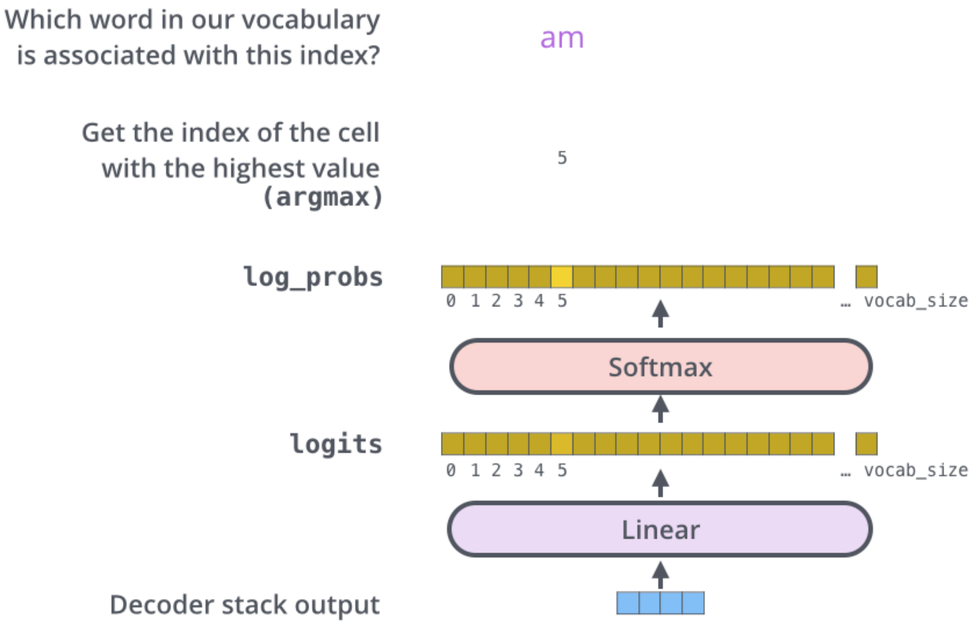
am

Get the index of the cell with the highest value (`argmax`)

5



Language Modeling Head



- it takes the output of the final transformer layer
- a linear layer L projects this output to logits (raw unnormalized scores) while "unembed" it at the same time:
  $L : \mathbb{R}^d \rightarrow \mathbb{R}^{|V|}$
- we use a softmax to transform the logits into probabilities over the vocabulary space
- we end up with a probability for each word in the vocabulary 💥


That's a
Transformer

Disclaimer

2018

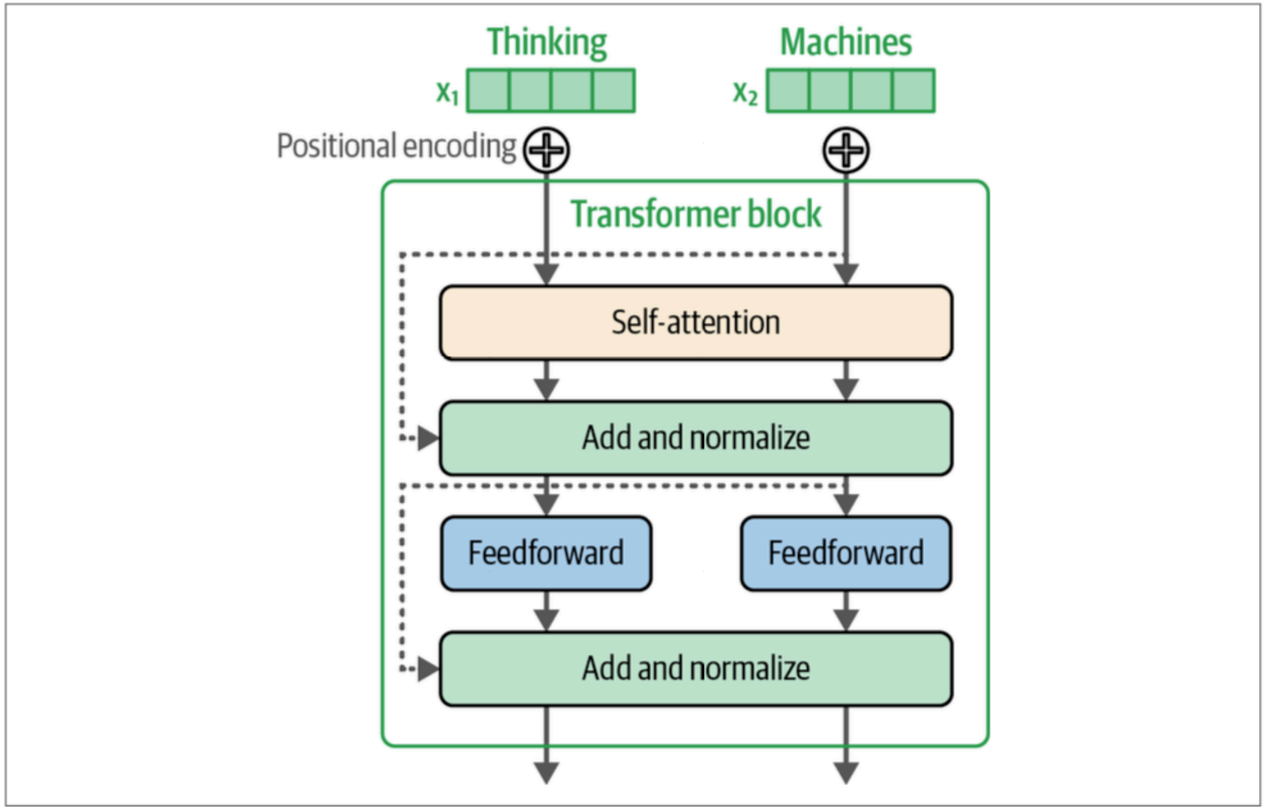


Figure 3-29. A Transformer block from the original Transformer paper.

2024

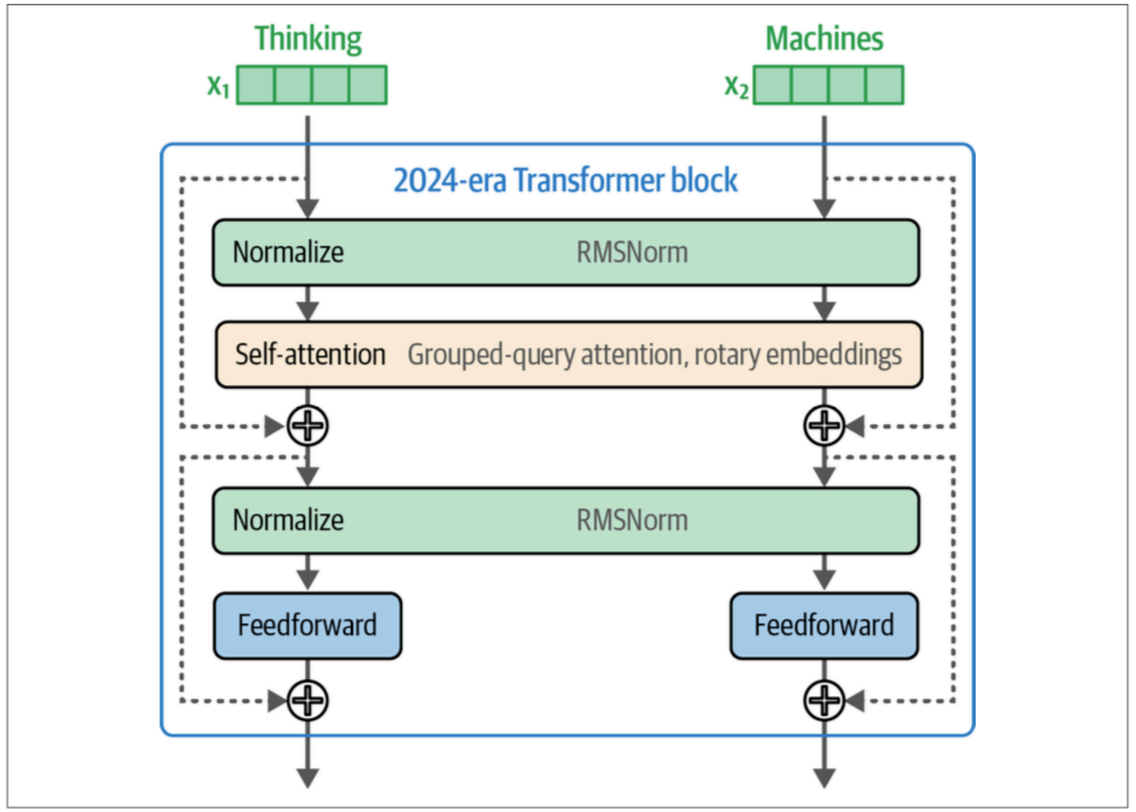


Figure 3-30. The Transformer block of a 2024-era Transformer like Llama 3 features some tweaks like pre-normalization and an attention optimized with grouped-query attention and rotary embeddings.

Conclusion

- 🫁 the Transformer architecture from 2018 builds the backbone of LLMs, until today
- 🗣️ self-attention allows “communication” between tokens
- 🚀 parallelization allows to speed up computations
- 🎨 several changes in architecture, training etc have been implemented since then

Next week: more about LLMs and GenAI!